A

Project Report on

Library Notifier

Submitted in partial fulfilment of the

Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

BY

Laxana Nilesh Tamke

Seat No. 3014290

Under the esteemed guidance of

Prof. Varsha Nikam.

DEPARTMENT OF INFORMATION TECHNOLOGY

HIRWAL EUDCATION TRUST's

COLLEGE OF SCIENCE (COMPUTER SCIENCE & INFORMATION TECHNOLOGY),

MAHAD –RAIGAD

(Affiliated to University of Mumbai)

NAAC Accredited with 'B+' Grade

Academic Year –: 2025-26

Hirwal Education Trust's
College of Science (Computer Science & Information Technology), At- Mahad-Raigad.
(Affiliated to University of Mumbai)
NAAC Accredited with 'B+' Grade
ISO Certified 9001:2015

Trust Estd.13 Sept. 2004
Regd.No. E-432-Raigad

AISHE CODE C-34101

Estd. 2009
College Code: 924

Shri. Kishor C. Dharia
Chairman

Mr. Sudesh N. Kadam
I/CPrincipal

Mumbai – Goa Highway,Behind Dharia Petrol Pump, At-Chambharkhind,Post / Tal. - Mahad, Dist- Raigad Pin Code: 402301
Cell No. 9011768522 | Website: https://hirwaleducationtrust.com | Email: hirwaleducationtrust@rediffmail.com

Project Certificate for B.Sc.(I.T) Student

This is to certify that the Project entitled "Library Notifier", is benefited work of **Laxana Nilesh Tamke** bearing **Exam No. 3014290** Submitted in partial fulfillment of B.Sc. (IT).The project has not been submitted for any other examination and does not form any other course undergone by the candidate.

It is further certified that they have completed all phase of the project.

Signature of Guide **Head of the Dept.** **Principal**

Internal Examiner **External Examiner**

DECLARATION

I, Laxana Nilesh Tamke, hereby declared that the project entitled “Library Notifier” submitted in the partial fulfillment for the award of Bachelor of Science in Information Technology during the academic year 2025-2026 is my original and the project has not formed the basis for the award of any degree, associateship, fellowship, or any other similar titles.

Signature of the Student:

Place: Mahad

Date:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, I am deeply thankful to my **Project Guide, Mrs. Varsha Nikam**, for their invaluable guidance, constant support, and insightful suggestions throughout the course of this project. Their encouragement and mentorship have been instrumental in shaping this work.

I would also like to extend my heartfelt thanks to **Mrs. Varsha Nikam**, Head of the Department, for providing the necessary resources and a conducive academic environment that greatly facilitated my research.

My sincere appreciation goes to all the **Faculty Members** of the department for their encouragement, feedback, and continuous support, which helped me stay focused and motivated.

A special thanks to my **friends and classmates**, whose constructive discussions and cooperation played a vital role during various stages of this project.

I am grateful to our respected **Principal, Mr. Sudesh Kadam**, for their support and for providing an institution that promotes academic excellence.

Lastly, I express my deep gratitude to **HET's CSCSIT** for giving me this opportunity to undertake and complete the project as part of my academic curriculum.

Thank you all.

Name: Laxana Nilesh Tamke.

Seat No: 3014290

INDEX

Sr. No	Title	Pg. No
1	**INTRODUCTION** 1.1 Background 1.2 Objectives 1.3 Purpose, Scope, and Applicability 1.3.1 Purpose 1.3.2 Scope 1.3.3 Applicability 1.4 Achievements 1.5 Organization of Report	2-4
2	**SURVEY OF TECHNOLOGIES**	5
3	**REQUIREMENTS AND ANALYSIS** 3.1 Problem Definition. 3.2 Requirements Specification. 3.3 Planning and Scheduling 3.4 Software and Hardware Requirements 3.5 Preliminary Product Description 3.6 Conceptual Models	6-12
4	**SYSTEM DESIGN** 4.1 Basic Modules 4.2 Data Design 4.2.1 Schema Design 4.2.2 Data Integrity Design 4.3 Procedural Design 4.3.1 Logic Diagrams 4.3.2 Data Structures 4.3.3 Algorithms Design 4.4 System Architecture 4.4.1 System Design 4.4.2 ER Diagram 4.4.3 DFD Diagram 4.4.4 Proposed Methodology 4.4.5 Class Diagram 4.4.6 User Case Diagram 4.5 User interface design 4.6 Security Issues 4.7 Test Cases Design	13-29
5	**IMPLEMENTATION AND TESTING** 5.1 Implementation approaches 5.2 Coding details and code efficiency 5.2.1 Code efficiency 5.3 Testing Approach 5.3.1 Unit testing 5.3.2 Integrated testing 5.3.3 Beta testing	30-34

	5.4 Modification and improvements 5.5 Test cases	
6	**RESULTS AND DISCUSSION** 6.1 Test reports 6.2 User Documentation	35
7	**CONCLUSION** 7.1 Conclusion 7.1.1 Significance of the system 7.2 Limitations of the system 7.3 Future scope of the Project	36-37
	REFERENCE	38
	GLOSSARY	39
	APPENDIX A	40-46
	APPENDIX B	47-49

GANTT CHART

Name of the phase	Expected Date of completion	Actual Date of completion	Signature of guide
Project Search	11-07-25	18-07-25	
Finalization & Allocation	14-07-25	21-07-25	
Investigating of System Requirement	01-08-25	08-08-25	
Data & Process Modeling	09-08-25	11-08-25	
System Design	21-08-25	09-09-25	
Program Design	10-09-25	19-09-25	
Program Coding & Unit Testing	25-09-25	27-09-25	
System Integration	15-10-25	15-10-25	
Project Review By Guide	11-11-25	18-11-25	
System Implementation	04-12-25	04-12-25	
Acceptance Testing	09-01-26	13-01-26	
Feedback for Improving Correcting & Modifying	21-02-26	11-02-26	
Final Report Writing & Documentation	10-03-26	23-02-26	

ABSTRACT

A manage library records, due dates, and fines is a common challenge in schools, colleges and other public libraries. Manual methods consume a lot of time, can lead to create struggle to ask for overdue books and calculate fines.

To solve this problem, the Library Fine Calculator and Due Date Notifier system is designed as an automated solution that make library operations easy. This system keeps records of members and books, allowing librarians to manage books give and take smoothly.

It's automatically sets due-dates and calculates the fines for late return book based on set rules. By tracking to give books, the system has less manual errors in fine calculation and ensures clarity.

Moreover, the project includes a notification feature that alerts members before the due date and tells them to return the books along with the fine amount. This can help to return the books in time to time.

1. INTRODUCTION

1.1 Background

In educational institutions, libraries are essential in providing students and instructors with access to learning materials, including books. Most libraries track their books and manage all records of book issues/returns manually (e.g., in registers or simple spreadsheets). This system of tracking creates many problems for libraries such as difficulties in tracking issued books, delays in finding overdue books, and the absence of adequate notification systems for overdue notices and reminders.

As the number of books and the number of users continue to grow, manually managing all of a library's operations will become increasingly less efficient and more time-consuming. The librarian may experience difficulties in maintaining accurate records, calculating overdue fines, and determining whether or not books are immediately available.

As the number of employees at a public library increases, so do the number of books and users. Students may also forget when their books are due and incur additional fines because they did not know until after the due date.

To resolve these issues, libraries need to invest in an automated library management system (LMS). Automated LMS will help improve the efficiency and productivity of libraries. One LMS that can be used to automate the processes of managing books, student records and Issuing and returning books is a web-based Library Notifier System (LNS). In addition to improving the management capabilities of libraries, LNS will also send notifications to library users when their books are overdue and will calculate the amount of overdue fines based on the number of days past the due date.

1.2 Objectives

1. To create a web-based solution to oversee the management of books and library records.

2. To create a uniform approach for issuing and returning library books by eliminating all manual processes.

3. To ensure accurate logs of student borrowing and inventory for library users.

4. To allow library staff to receive notifications when borrowers exceed their borrowing limit.

5. To automatically calculate fines for late returns of library materials.

6. To enhance efficiency and organization within a library.

1.3 Purpose Scope, and Applicability

1.3.1 Purpose

Library Notifier is a web application designed to manage and automate the library's various activities, including adding books, issuing and returning books. It provides an effective means for keeping track of the number of books you have on hand and possesses capabilities for notifying you when a book is overdue, and calculating a fine for any late returns. The Library Notifier improves both the efficiency of your library management, as well as reducing the amount of manual work involved in managing those processes, allowing for a better-organized and more reliable library system.

1.3.2 Scope

- Supervise library books and their records.
- Cypher and receipt of the library's books.
- Control, all books in your library's possession.
- Contact users with overdue items.
- Automatically calculate costs for users who return items late.
- Control student / user activity records.
- Improve the efficiency of the management of the library.

1.3.3 Applicability

- Schools and educational institutions often use this software to maintain library records and student ID's.
- To find ways for educational organizations to remain updated regarding their issued and returned books.
- Aiding in the book inventory management of the library.
- Training centers and coaching institutions that have limited library holdings could also use this program.
- Completely eliminates the need for cumbersome record-keeping and paper documents.
- Used for automatically monitoring overdue books and fees/fines.
- Provides a more organized environment in computing and managing all aspects of library operations.

1.4 Achievements

- A Web-based Library Notifier System was designed to automate library operations.
- Book Management Has Been Successfully Completed (Adding, Viewing and Updating Books).
- A Book Issue and Return System has been Designed with Complete Record Tracking.
- A Student Management Module was Developed to Capture Student Information.
- Automatic Notifications are sent out for Overdue Books.
- Late Returns are subject to Fine Calculation.
- A Simple Interface Design Has Been Implemented for Simple Library Administration.
- Automation has Increased Efficiency and Accuracy Compared to Manual Library Administration.

1.5 Organization of Report

Chapter 1: The Introduction:- The introduction consists of background information about the project and its objectives and purposes; it also includes the defined scope of the project.

Chapter 2: Survey of Technologies:- As a research-based chapter, this chapter focuses on the technologies utilized for developing the system, such as Python, Flask, HTML, CSS, and a MYSQL database.

Chapter 3: Requirements and Analysis:- This chapter explains the problem definition, system requirements, and analysis of the features needed to effectively manage a library.

Chapter 4: System Design:- This chapter outlines the overall system architecture and the database design, module structure, and user interface design of the system.

Chapter 5: Implementation and Results:- In this chapter, implementation details such as back end and front end development, database integration, and overall system functionality are provided.

Chapter 6: Conclusion and Future Work:- This chapter summarizes the conclusions derived from the implementation of the project in addition to the recommendations for potential future enhancements of the system.

2. SURVEY OF TECHNOLOGIES

- **Programming Language :-** Python.
  It was selected because it is popular, relatively easy to learn, and support fast development.

- **Database Management System (DBMS)** :- MySQL.
  Provides secure data storage, retrieval, and update operations.

- **Frontend Technologies** :- HTML, CSS.
  Delivers a user-friendly interface design for librarians and members. Guarantee that the system is easily accessible and responsive.

- **Backend Framework** :- Flask (Python Framework).
  Manages requests between the user interface and the database. Assists in building secure and scalable web applications.

- **Notification Service** :- Automated Overdue Notification
  The system detects overdue books and displays notifications with fine calculation, helping the administrator track delayed returns easily.

- **Development Tools** :- VS Code.
  VS Code is used as the development environment for writing, testing, and debugging the application code.

- **Version Control System** :- Git & GitHub.
  These tools help manage different versions of the project and allow easy tracking of changes during development.

- **Operating System** :- Windows / Linux.
  These operating systems provide the platform to develop, run, and test the web application.

3. REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

- **Context:** Many libraries manage their records(listings) and due dates for books by hand or with limited computer systems. Every year the number of students that forget when their books are due and therefore return their books late both creates a problem for libraries and provides difficulty for libraries that are trying to keep track of what they have issued to students.

- **Problem:** The percentage of people who do not return books when they are supposed to causes an immense amount of use to libraries, as well as incorrect usage of library inventory.

- **Goal:** The project the Library Notifier System will develop a web based application that will manage records of books, track books that have been issued to students and notify both libraries and students as to the due date of their book.

- **Key Challenges:** Ensure accurate book tracking, timely notifications of overdue books, timely management of student records and keeping the user interface of the web application and database synced with one another.

3.2 Requirements Specification

The Library Notifier System is designed to simplify library operations by tracking member and book records, book check-outs, and book check-ins with the ability to calculate due dates automatically, along with the calculation of fines for overdue books.

The system will also notify members regarding their due date and overdue books via email/SMS, which promotes timely returns. The system will be intuitive, dependable, and secure, with a simple interface, accurate calculation of fines, and secured data access. It will work on Windows and Linux and can scale from small to large libraries.

The system will be programmed in Python - with frameworks like Flask, database technology like MySQL, and front-end technologies - HTML, CSS; and versioning control will be done through Git and GitHub. The minimum hardware requirement is an Intel i3 processor with electrical DRAM of 4 or more and a minimum of 250 GB of storage. For smooth multi-user operation we recommend a computer with an Intel i5/i7 processor, electronic DRAM of 8 or more, and a minimum of 500 GB of storage.

- The system will help to eliminate human error by calculating fines and due dates automatically.
- The system increases efficiency by sending out notifications to members so that books can be returned in a timely manner.

3.3 Planning and Scheduling

During the system design phase, the design of the Library Notifier Web Application is focused on organizing and outlining how everything will be developed before you start working on it. This phase also makes sure that each of the system's components are clearly outlined, including how to manage books, how to manage students, how to manage notifications, and what operations will occur with the database. This phase will therefore identify any resources needed to build the system and establish timelines for each development stage.

Planning Activities :-

- **System Architecture Design:-**
  - Develop the overall architecture of the web application (i.e., what will the front end look like, what will the back end look like, and how will the database be designed).
  - Identify major modules of the application (i.e., book management, student management, book issuance and book return, notification).
- **Technology and Tool Selection:-**
  - Identify the appropriate technologies for the development of the systems (e.g., Python/Flask for the back end, HTML/CSS/JavaScript for the front end, and SQLite for database management).
  - Identify tools that will be used to develop and debug the application (e.g., Visual Studio Code).
- **Data Flow and Inter-module Interaction:-**
  - Plan how data will be moved throughout the system (e.g., the book data, student data, and book issued data are).
  - Identify how different modules will interact with one another (e.g., how the book issuance module will update the database and trigger notifications).
- **Define Input and Output Specifications:-**
  - Input includes book information, student information, and data concerning lending/returning books entered by admins.
  - Output consists of displaying book records (such as those that were issued) and those that are available, sending overdue notifications, and calculating fines.

Scheduling Activities:-

- **Establish Design Timeline:**
  - Design timetable to include how long it will take to design each module (such as designing the database, user interface, book management system, and notifications).
- **Set Design Milestones:-**
  - Completion of database schema design and system architecture design; Completion of structure for each module and layout of user interface; Integration of front-end/back-end/database.
- **Resource Planning:-**
  - Identify each person's job and what aspect of development they'll work on (i.e., front-end designer, back-end developer, database administrator).
  - Identify all the software tools and systems required for development and testing.

3.4 Software and Hardware Requirements

3.4.1 Software Requirements

The development and execution of the Library Notifier Web Application require certain software components that support coding, database management, and system operation.

- **Operating System:**
  - Windows / Linux operating system is required to develop and run the application.
- **Programming Language:**
  - Python is used for implementing the backend logic and handling server-side operations.
- **Web Framework:**
  - Flask framework is used to develop the web application and manage communication between the user interface and the database.
- **Frontend Technologies:**
  - HTML, CSS, and are used to design the user interface and ensure a user-friendly experience.
- **Database Management System:**
  - MySQL database is used to store and manage data such as books, students, and issued book records.
- **Development Environment:**
  - Visual Studio Code (VS Code) is used for writing, editing, and debugging the application code.
- **Web Browser:**
  - Google Chrome or any modern web browser is required to run and test the web application.
- **Version Control System:**
  - Git and GitHub are used to manage source code versions and track changes during development.

3.4.2 Hardware Requirements

The Library Notifier Web Application requires basic hardware components to develop, run, and test the system efficiently.

- **Processor:**
  - Intel Core i3 or higher processor for smooth system performance.
- **RAM:**
  - Minimum 4 GB RAM is recommended for running the development environment and web application.
- **Storage:**

- At least 20 GB free disk space for installing software, storing the project files, and database.

• **Display Monitor:**
- Standard monitor with minimum resolution of 1366 × 768 for proper user interface display.

• **Keyboard and Mouse:**
- Required for user input and system interaction.

• **Internet Connection:**
- Required for downloading libraries, frameworks, and accessing version control systems such as GitHub.

3.4.3 Functional Requirement

The System should have the ability to:

1. Create, change and administration of library records of books
2. Keep and manage students' data such as name, roll number and class.
3. Book program for the student and record when the book is issued and returned
4. Book return program and update books available.
5. Overdue detection of books and provide notification of overdue along with computation of fees.
6. Show records of books, books issued, and notification on web page.

3.4.4 Non-Functional Requirement

1. A user-friendly interface is provided to allow for simple library administration.
2. The system responds quickly when performing functions such as adding, issuing or checking out books, and looking up records.
3. The database maintains the confidentiality of data for both book and student records.
4. The system operates consistently with very few errors in normal use.
5. This system is designed to be easy to maintain and expand in the future.
6. Access to the system can be made through standard web browsers without requiring special software.

3.5 Preliminary Product Description

The Library Notifier web application will be an online software solution specifically to manage the library and help automate many of the day-to-day functions involved with library operations. The Library Notifier will enable administrators to keep their records for all books, student information, and all other functions for issuing and returning of books in an easy manner.

A central website to house all library information will be provided through the Library Notifier web application. The application will provide various modules such as book management, student management, book issuing and returning, and library notification system. Administrators will have the ability to add/additions of new books to the library, update existing book titles, and also check available books in the library at the same time.

In addition, the Library Notifier will allow administrators to maintain student records relating to borrowed books and also create procedures for maintaining a complete history of borrowed books in the library. The Library Notifier will also have a very effective notification system that will provide administrators with overdue information on borrowed books and late fines due. This information will allow library administrators to keep track of any overdue books, therefore maintaining the proper discipline of the library.

The backend processing of the system has been created using Python along with the Flask Framework and on the front end HTML, CSS will be used as a user interface to interact visually with the application. The back end database will utilize MySQL in order to manage and store all necessary data. In addition to its backend capabilities, the application will provide a simple, easy-to-use interface that allows for an efficient way to manage libraries and decrease manual recordkeeping's limitations.

3.6 Conceptual Models

Here are some high-level models that explain how the system works:

- **Module Design Diagram:** the major components of the library system, which include (but are not limited to) book management, student management, book issuing and returning, notifications, and database modules.
- **System Design Diagram:** The overall system architecture including (but not limited to) the frontend interface, backend server, and database.
- **Data Flow Diagram (DFD):** Represents how data (including but not limited to) book details, student information, and transaction details for issues/returns flows between the different system modules.
- **Entity-Relationship Diagram (ERD):** The relationships between the various entities in the database, including books, students, issued books, and notifications.

- **Schema Design Diagram:** Represents the structure of the data in the database, including tables, fields, and relationships.
- **Use Case Diagram:** The interaction between the administrator and the system regarding such actions as adding books, issuing books, returning books, and viewing notifications.
- **Proposed Methodology:** The methodology used to develop the library notifier web application, including requirement analysis, system design, system implementation, system testing, and system deployment.

4. SYSTEM DESIGN

4.1 Basic Modules

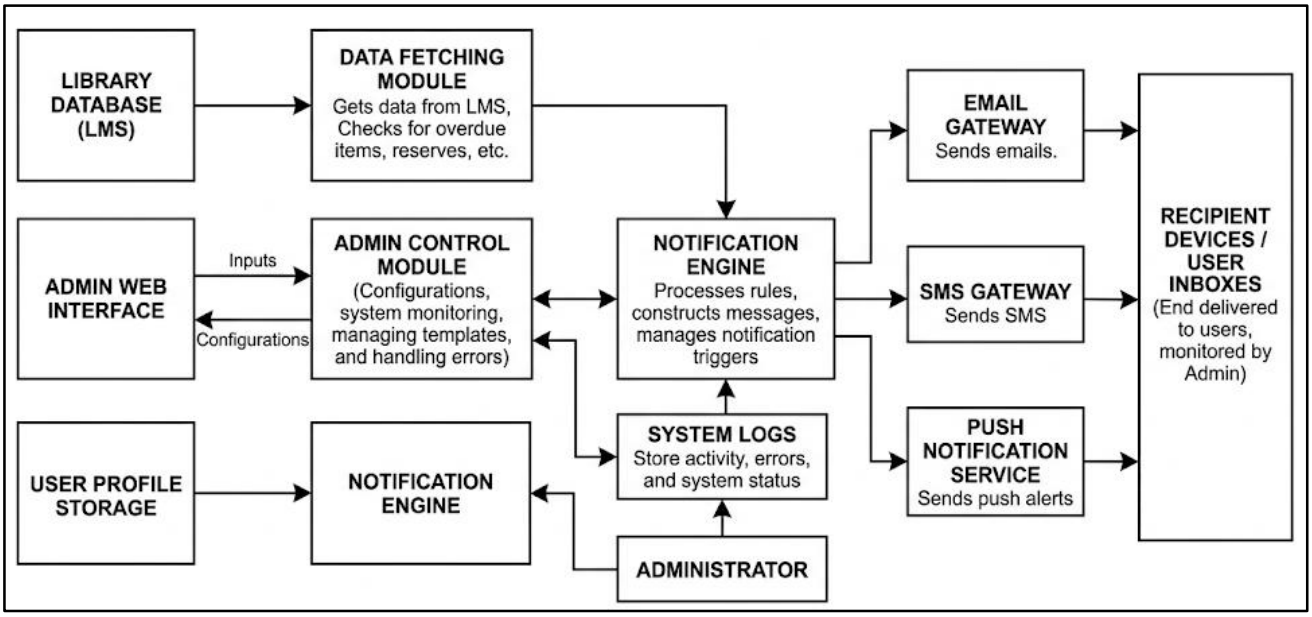


Fig 4.1 Basic Modules

The diagram shows modules of the library system, each with a function based on books, issue/return, fees, notifications, etc., to ensure the smooth operation of the overall service.

4.2 Data Design

4.2.1 Schema Diagram

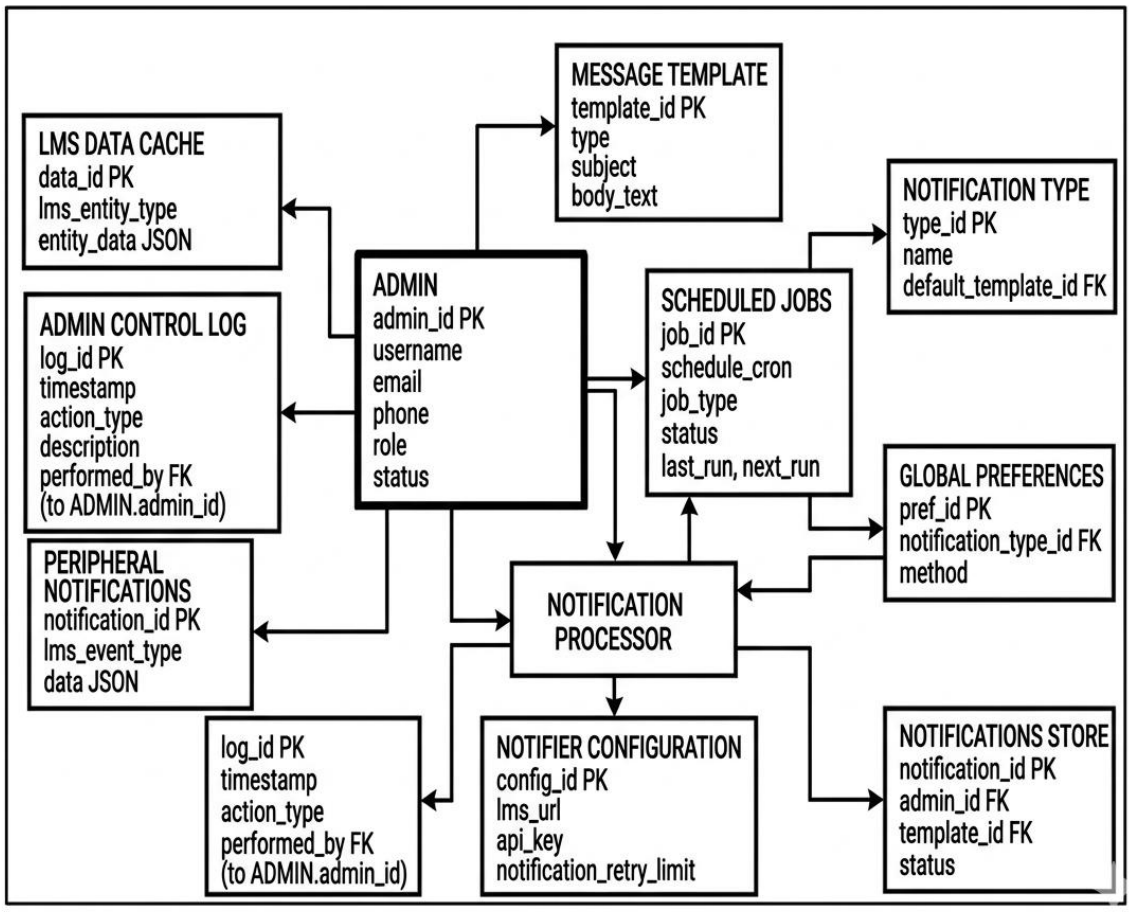


Fig 4.2.1 Schema Diagram

This schema diagram illustrates the database tables and relationships that connect members to books, transactions, and notifications. This helps to ensure data consistency and accuracy.

4.2.2 Data Integrity and Constraints

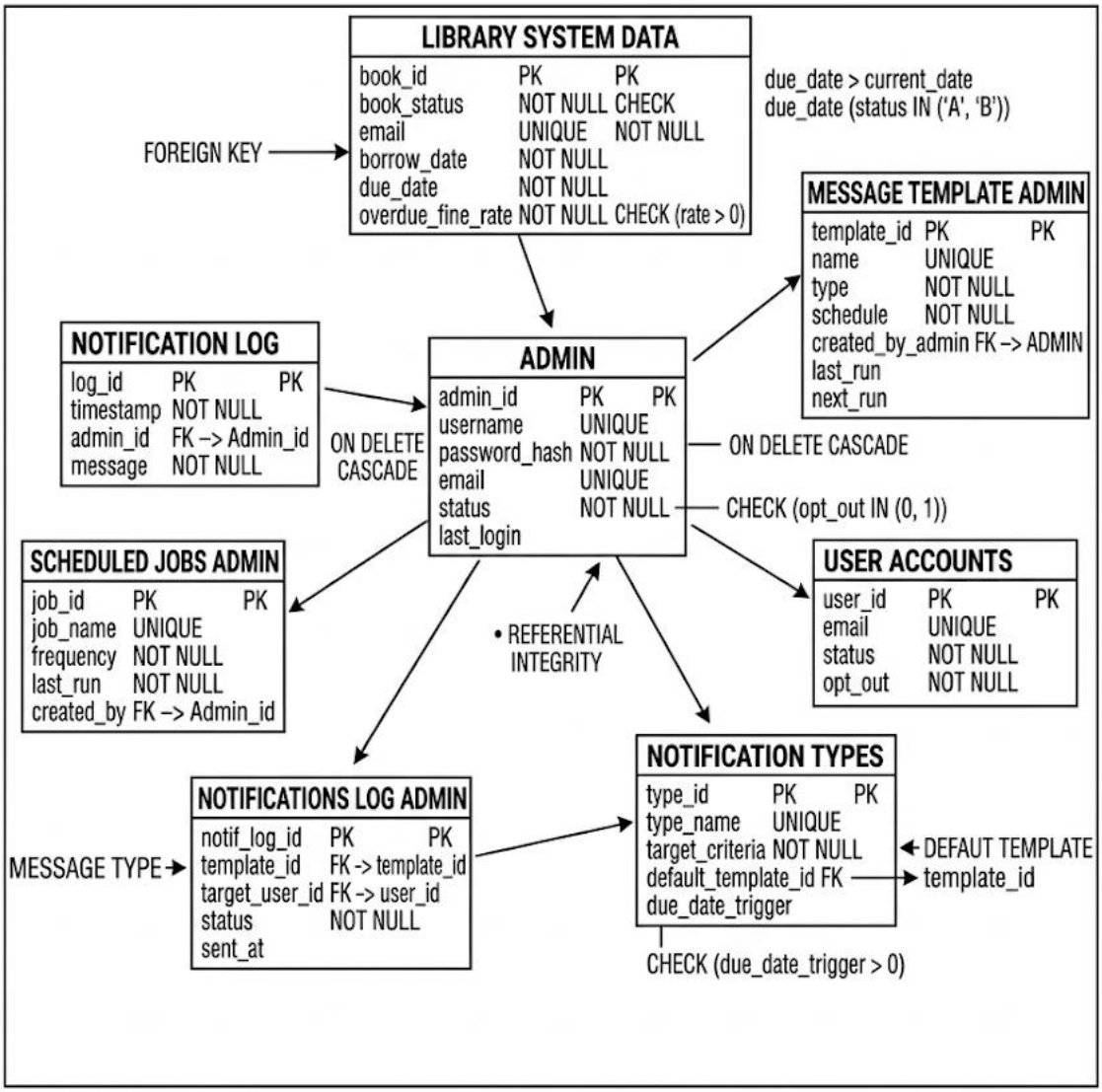


Fig 4.2.2 Data Integrity and Constraints

Data integrity and constraints guarantee accurate and consistent data within the system through rules like primary keys, foreign keys, unique values, and validation checks.

4.3 Procedural Design

Procedural design refers to outlining the sequence of processes and logical progression used by a system to perform its operations. For example, in the Library Notifier Web Application, there are defined steps and processes associated with adding books, issuing books, returning books, calculating fines, and generating notifications, all of which are performed in an orderly fashion so as to maintain consistency and accuracy with regard to library record management.

The Library Notifier Web Application is an example of a system designed using procedural design principles. The Library Notifier Web Application consists of a series of distinct procedures or functions, each responsible for different functions such as adding or updating information about books (via the book management module) or maintaining student information (via the student management module) or processing the issuance and returning of books (via the issuing and returning of books module). When the system is processing the issuance of a book, it will first check whether the book is available in inventory; once determined that the book is available, the system will record the details of the issuance of the book, assign a due date for the return of the book, and update the corresponding inventory information for the book in the database.

The book-returning process operates similarly to the book - issuing process. When a book is returned to the library, the application verifies the record of the book - issuance, updates the book - status in the database, determines if any fee applies to late book returns, and calculates any applicable fees owed for the book; then, the notification module creates an alert for any books with overdue status so that the library administrator can take action on the overdue book. These functional processes ensure that the flow of information among the various modules of the application remains clear and controlled.

Finally, because of the modularity of the application, functional processes can be reused, as an example; the application uses the same processes for calculating and collecting late fees for overdue books.

4.3.1 Logic Diagrams

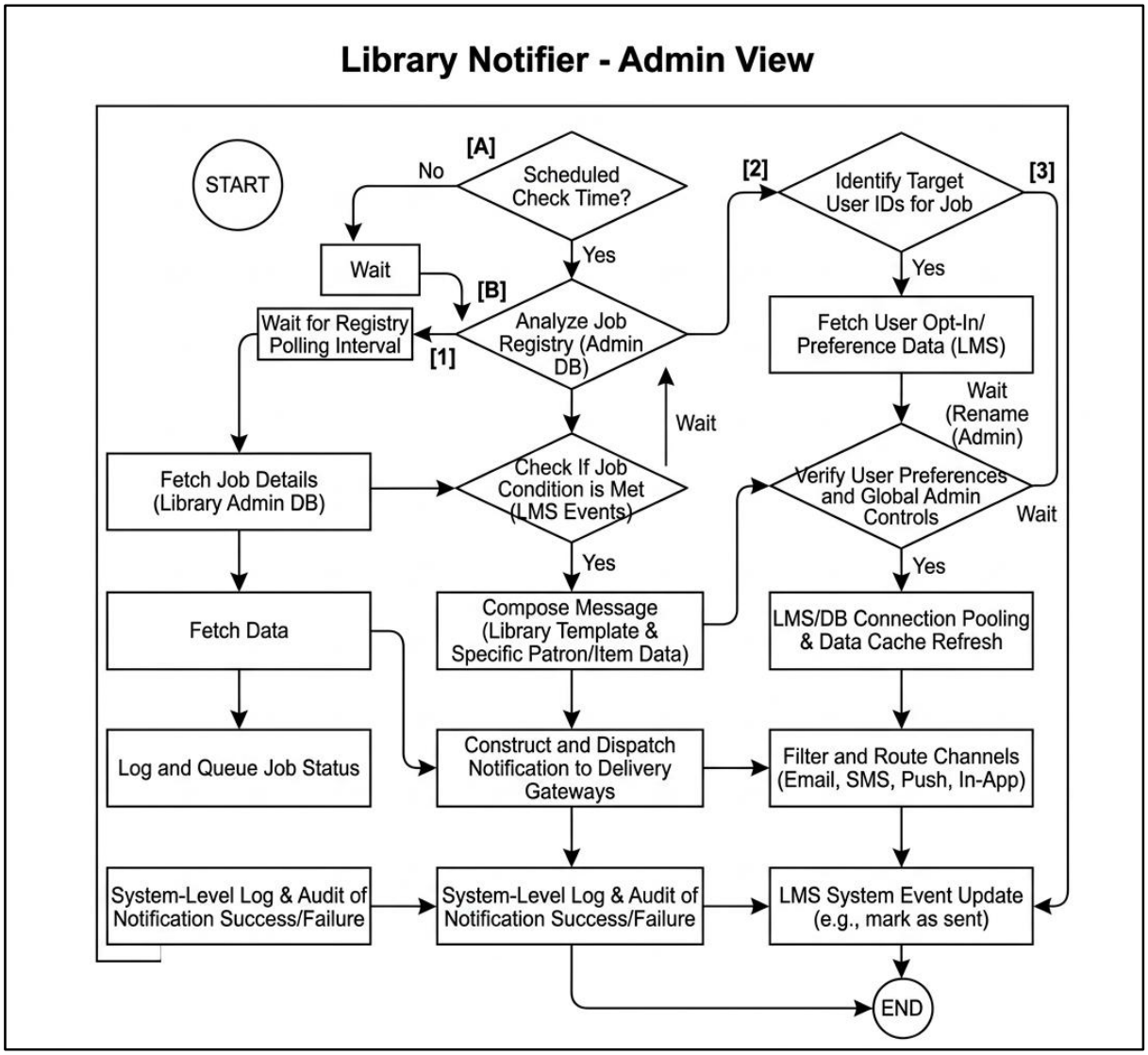


Fig 4.3.1 Logic Diagram

The logic diagram illustrates the sequence of operations and decisions in the system, like issuing a book, computing fees, and notifying users.

4.3.2 Data Structures

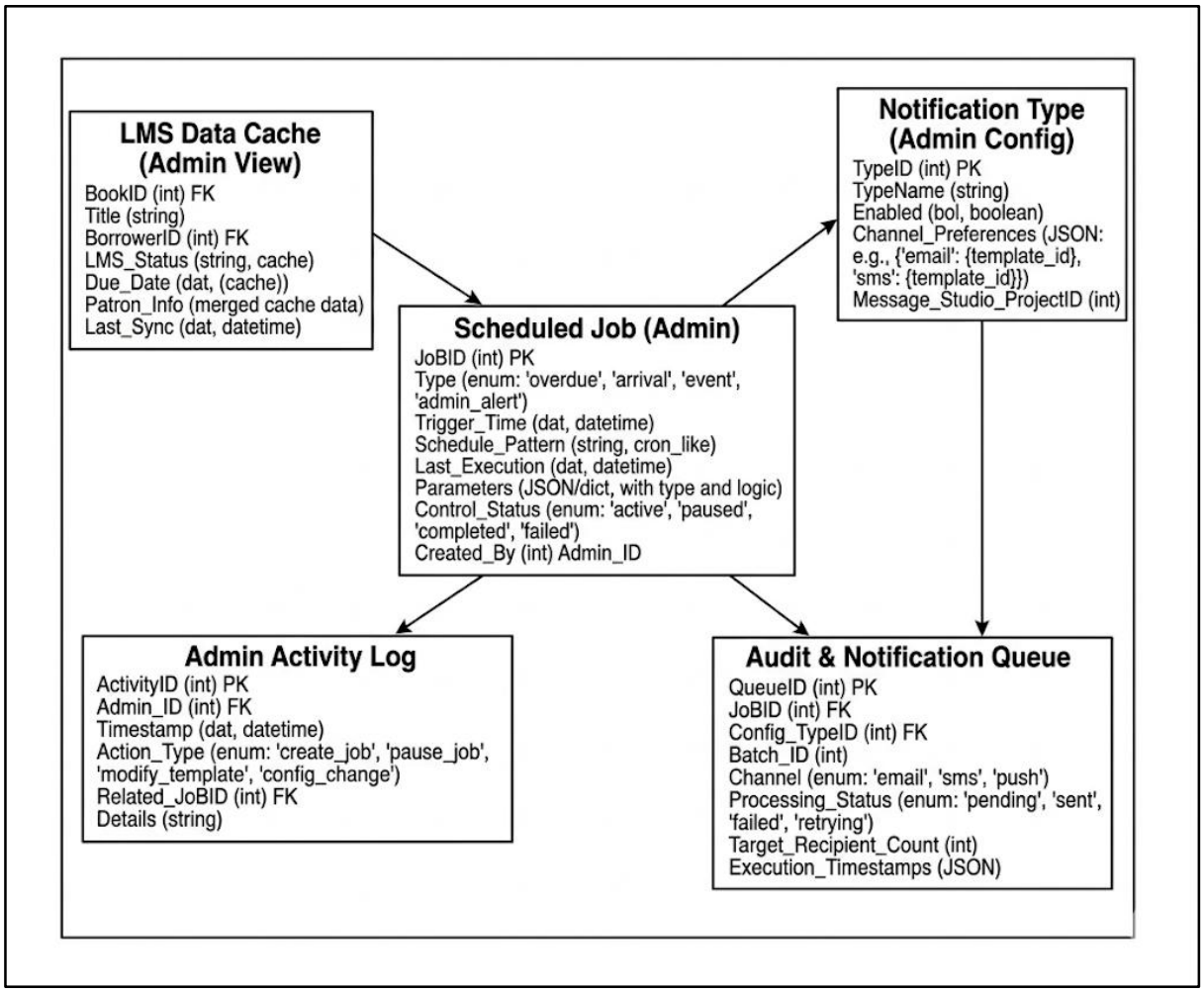


Fig 4.3.2 Data Structure

Data structures serve the purpose of organizing and storing data within the system, such as members, books, transactions, and notifications, for efficient access and management.

4.3.3 Algorithms Design

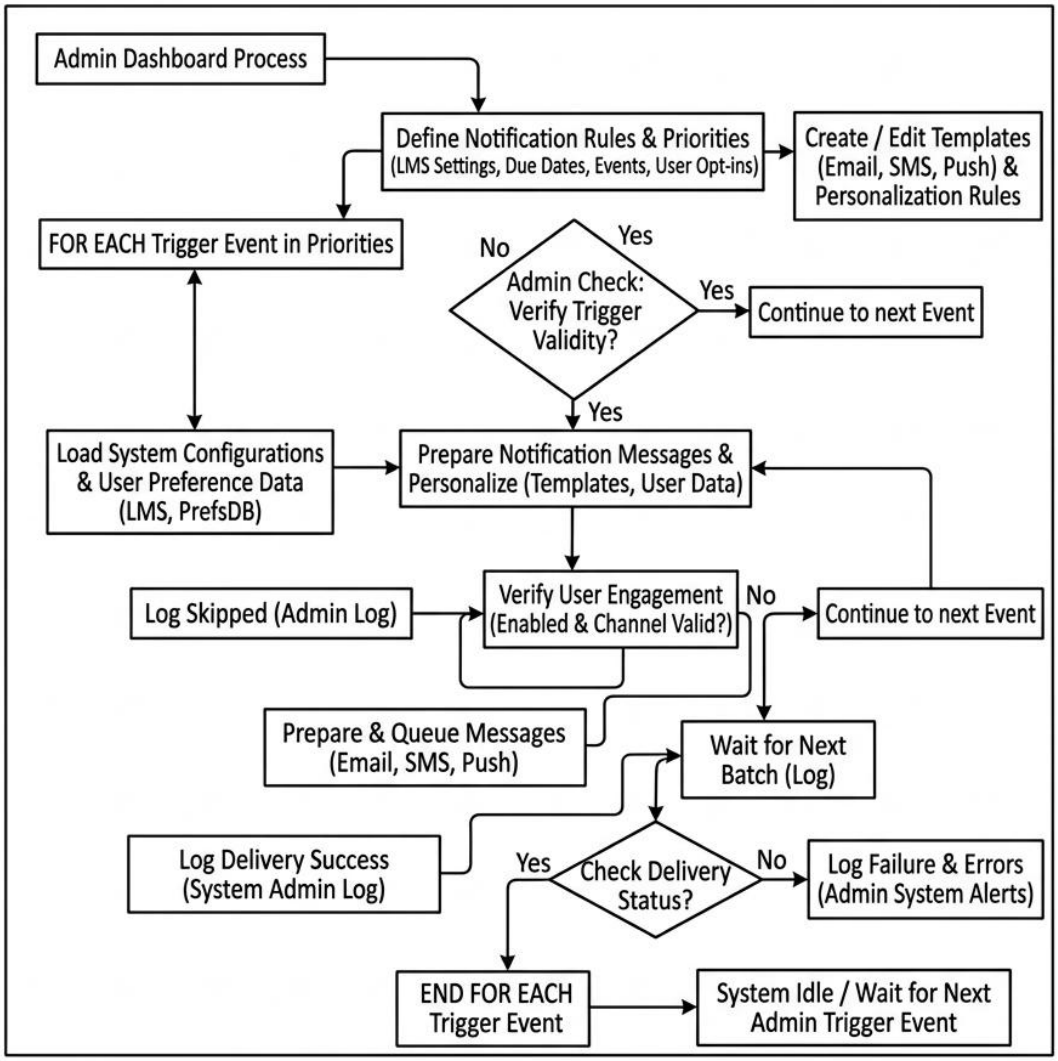


Fig 4.3.3 Algorithms design

Algorithm design defines the step-by-step logic for tasks like issuing/returning books, calculating fines, and generating notifications.

4.4 System Architecture

4.4.1 System Design

- **System Design**

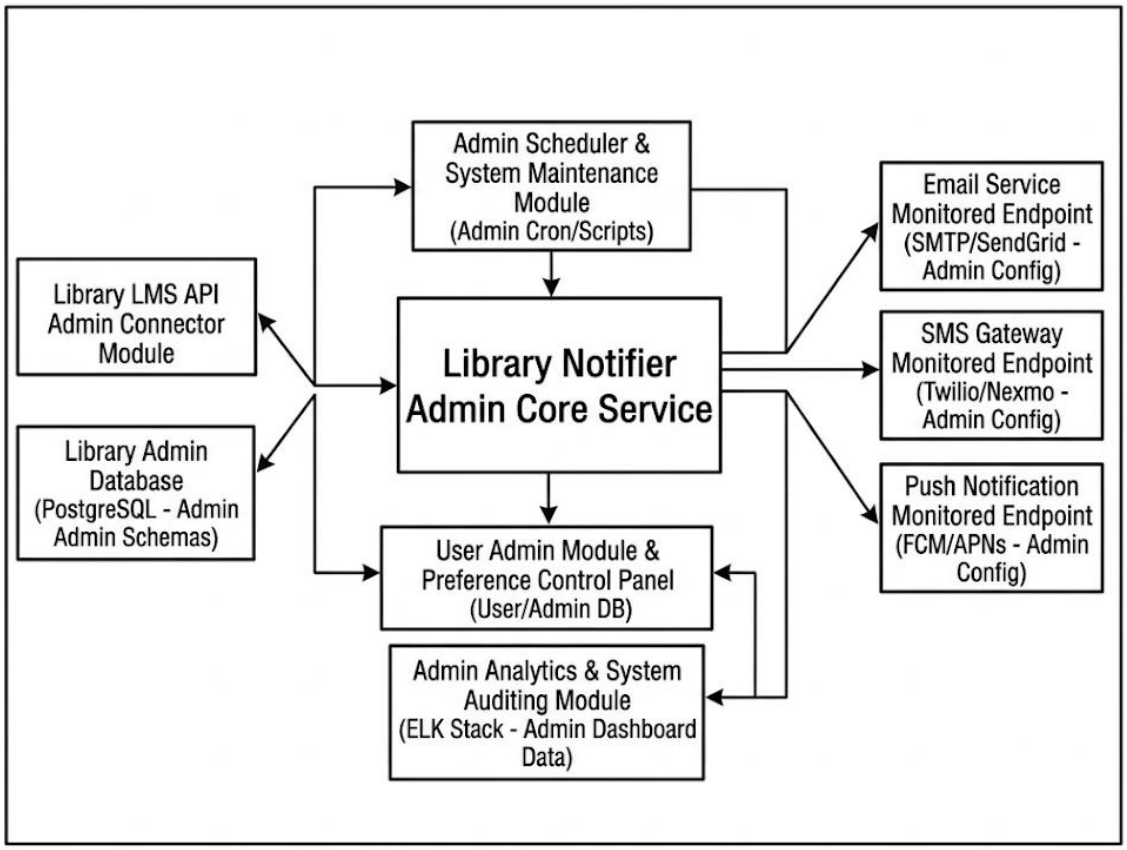


Fig 4.4.1 System Design

Internal system design demonstrates how modules, data, and processes interact within the system in a way that allows library operations to be successfully performed.

- **System Overview Design**

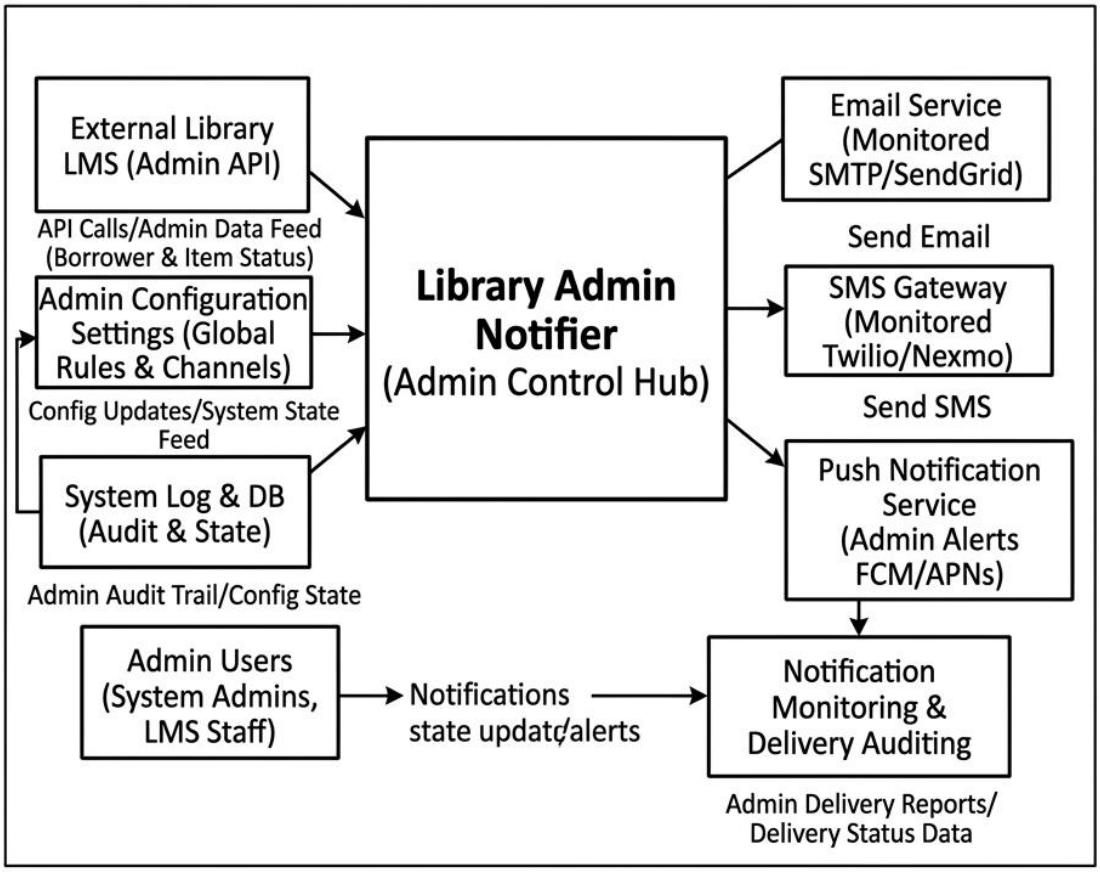


Fig 4.4.1 System Overview Design

External system design demonstrates the interaction of the system with users and external services including librarians and members and the delivery of notifications.

4.4.2 ER Diagram

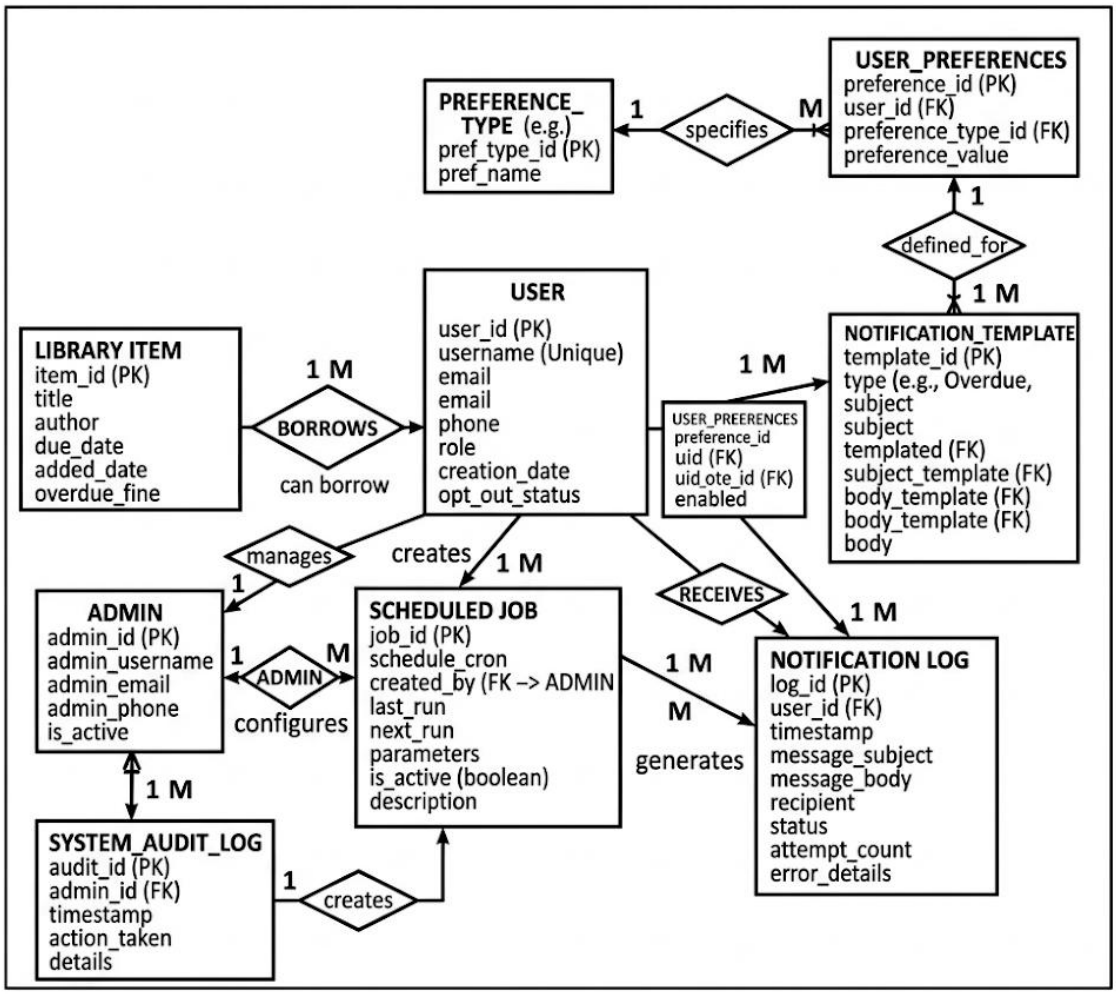


Fig 4.4.2 ER diagram

The ER diagram depicts entities such as Members and Books, Transactions and Notifications, and their associations in the library system.

4.4.3 DFD Diagram

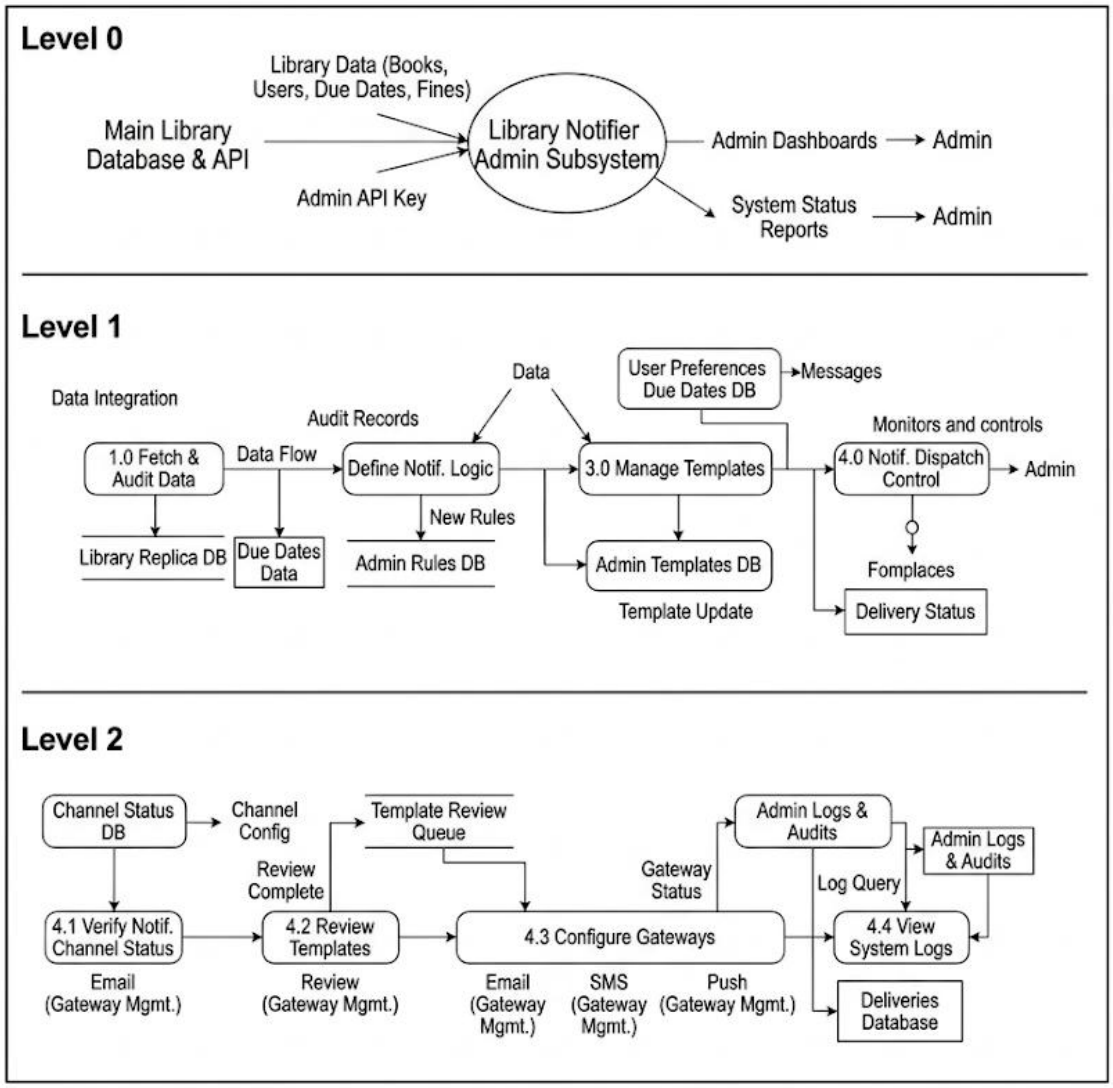


Fig 4.4.3 DFD diagram

The DFD diagram displays the movement of information between users, processes, and data stores of the library system, for example, issuing books, calculating fines, or sending notifications.

4.4.4 Proposed Methodology

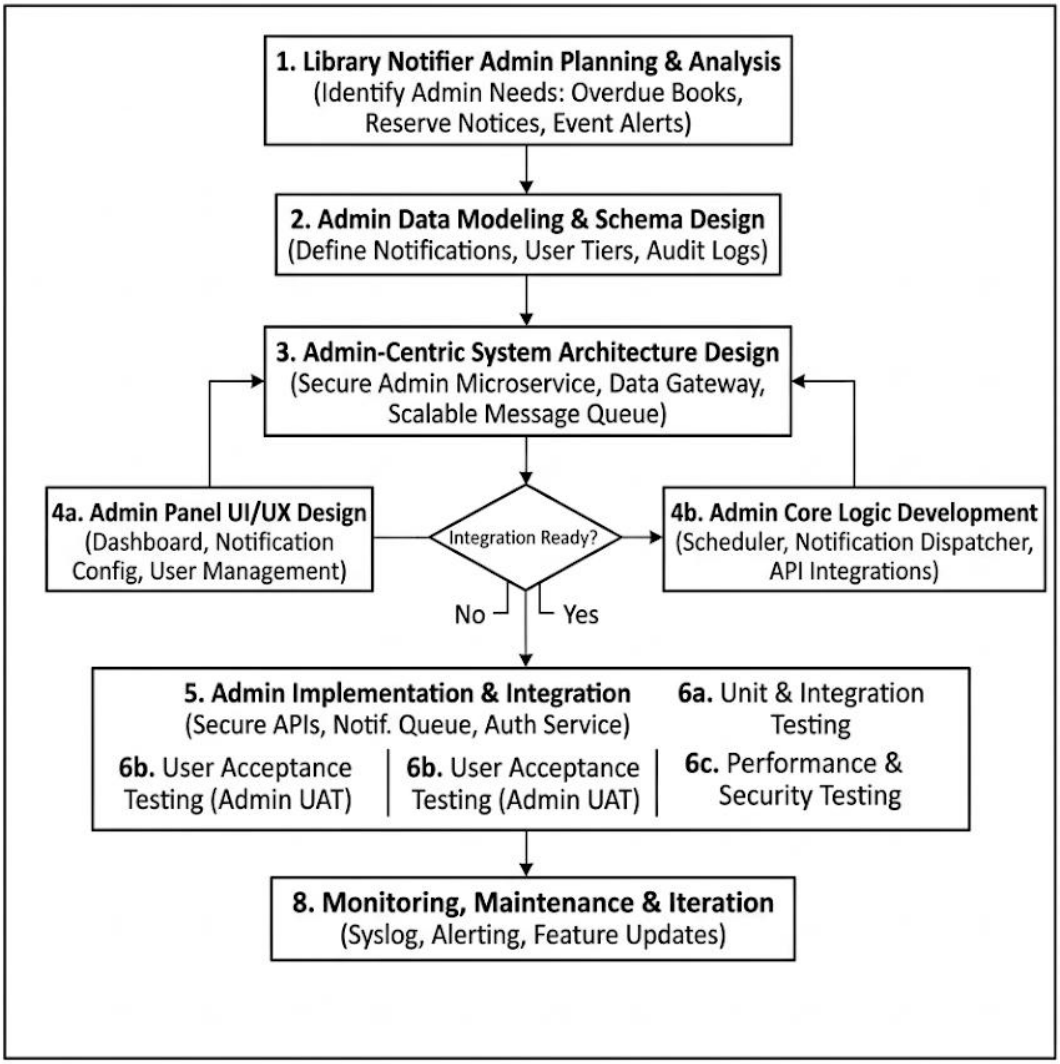


Fig 4.4.4 Proposed methodology

The proposed methodology discusses the flow from circulating books to the calculation of fines and notifications, ensuring a smooth and accurate running of library operations.

4.4.5 Class Diagram

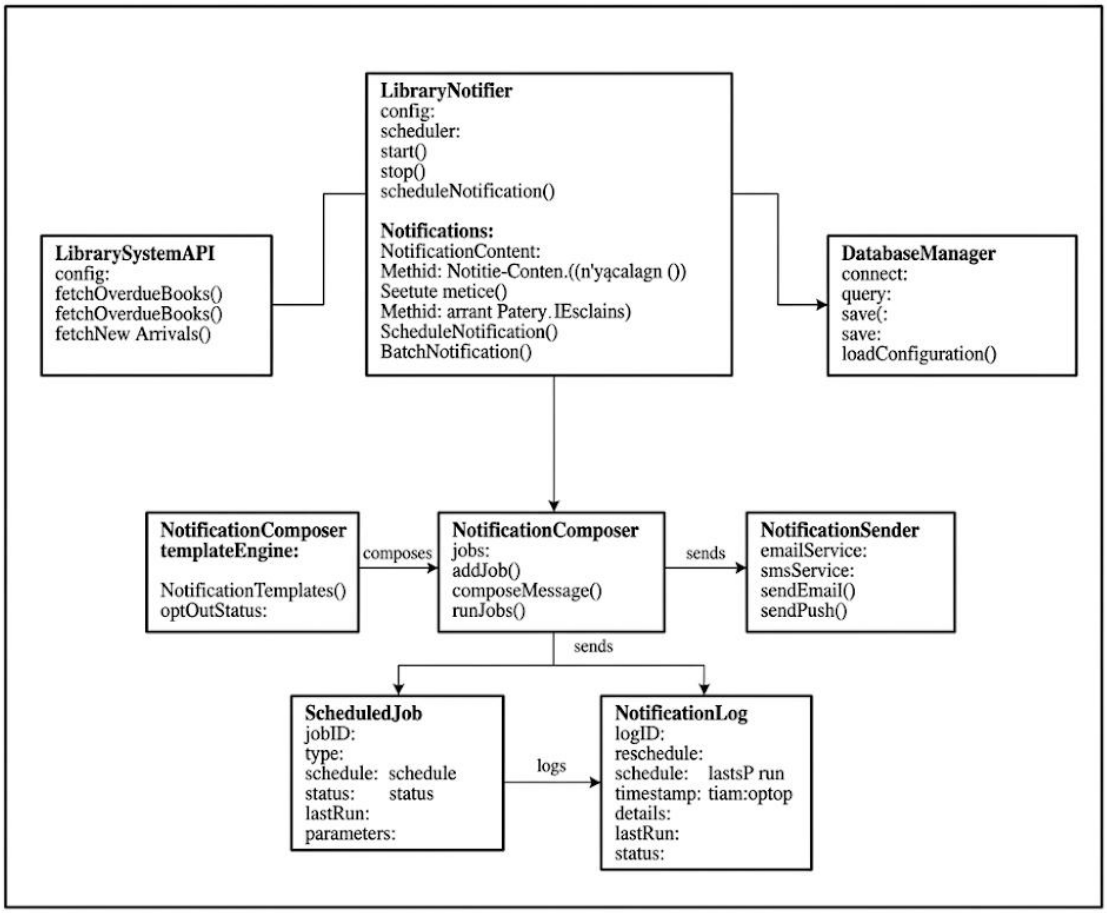


Fig 4.4.5 Class diagram

The class diagram describes the system classes of the Member Class, Book Class, Transaction Class, and Notification Class, with the attributes, methods, and relationships of each class.

4.4.6 Use Case Diagram

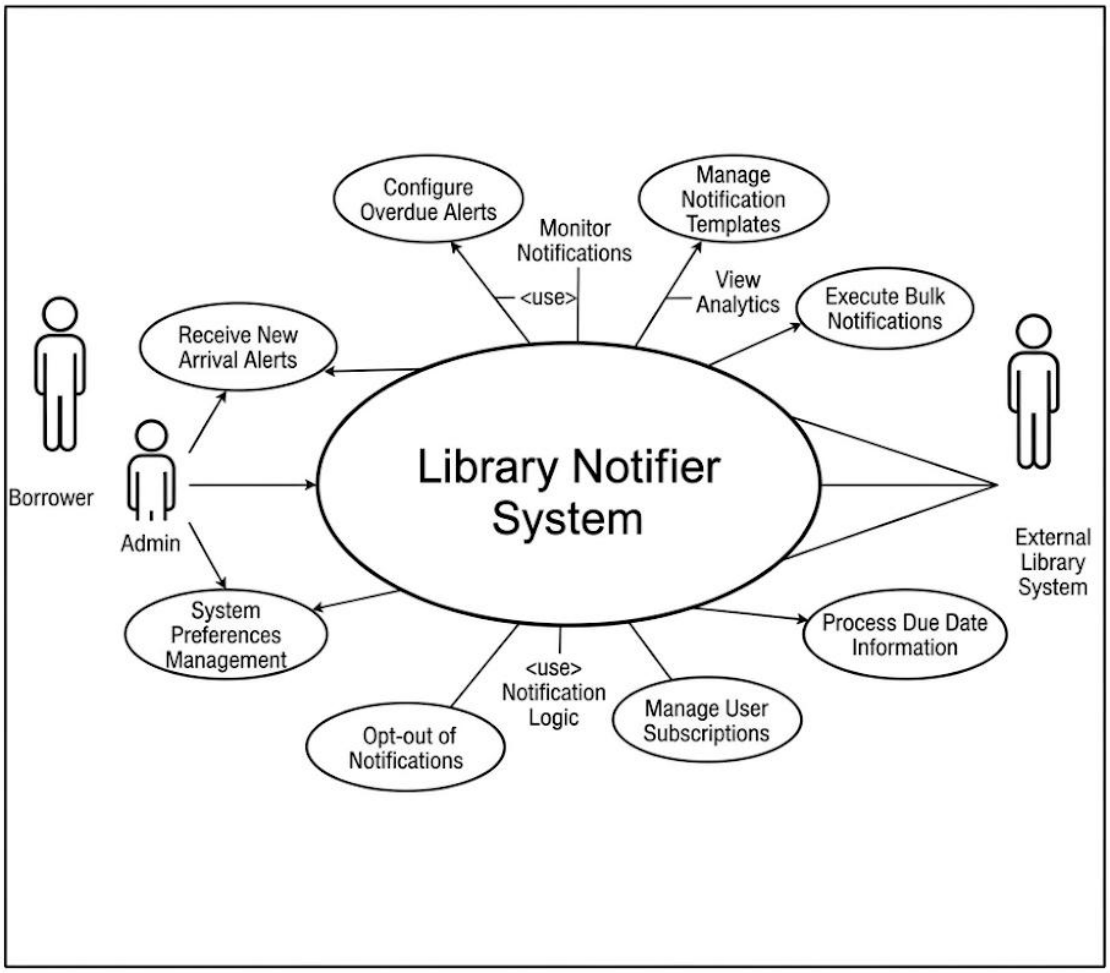


Fig 4.4.6 Use case diagram

The use case diagram provides information on the actors (Librarian, Member) and interactive actions between the actors and system, including issuing/returning books and notifications.

4.5 User interface design

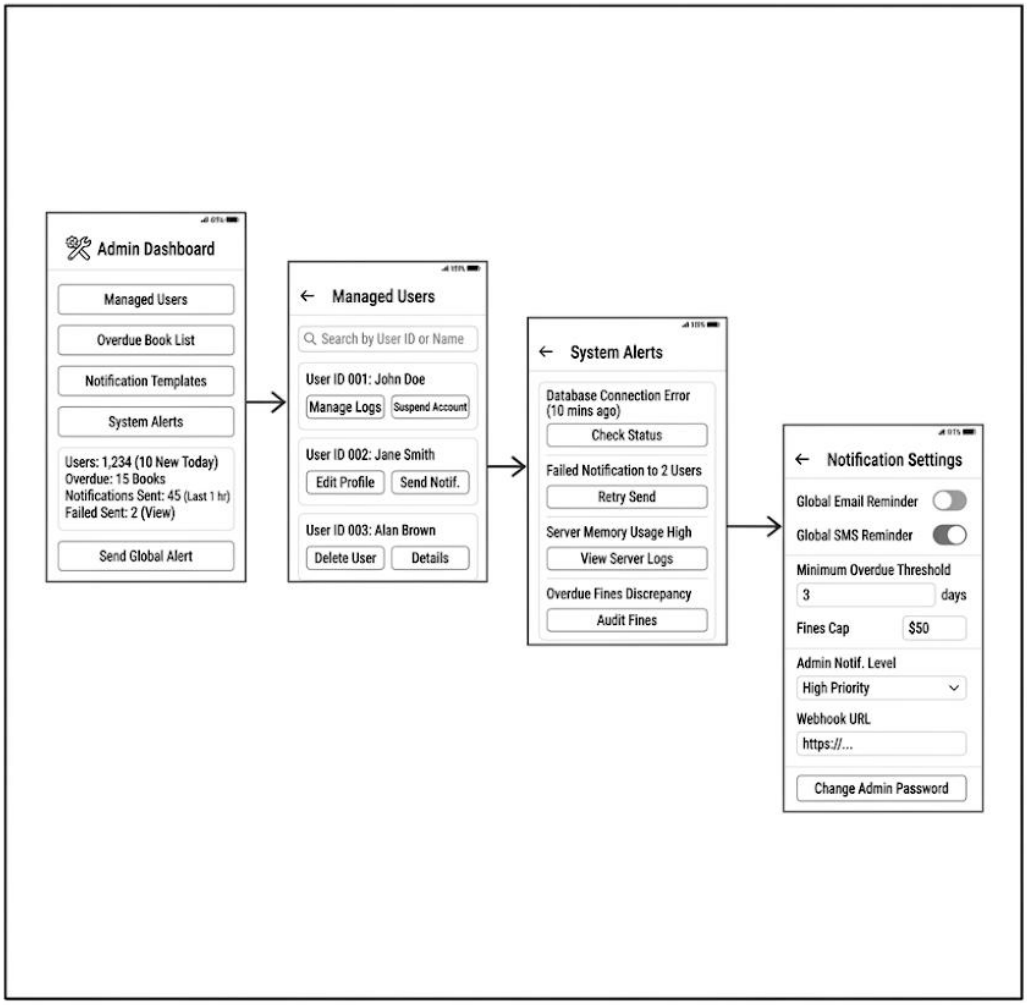


Fig 4.5 User interface design

The user interface design illustrates the configuration and organization of screens that librarians and members use to engage with the setup.

4.6 Security Issues

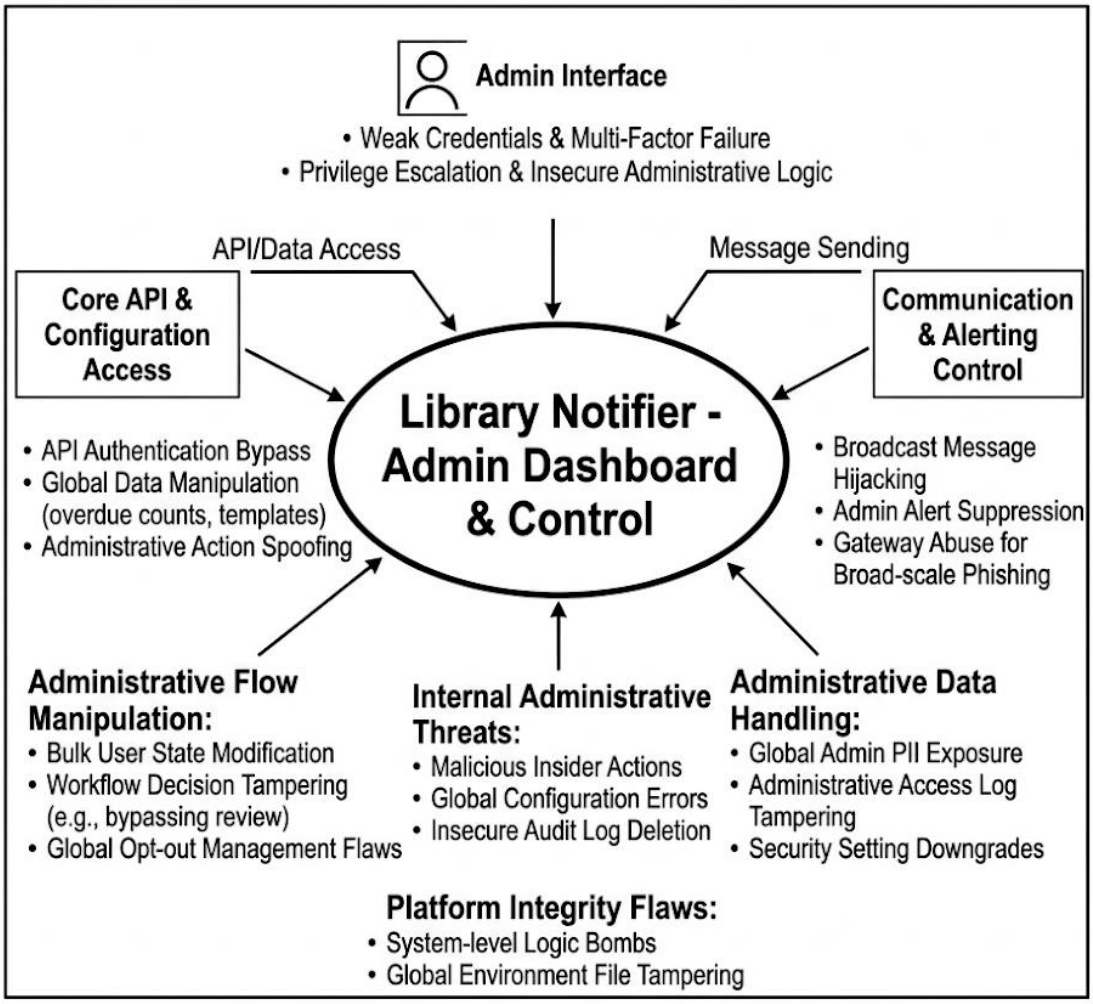


Fig 4.6 Security issues

Security issues illustrate possible threats to member data, book records, and associations' fines, and the safeguards meant to protect against unauthorized access or usage.

4.7 Test Case Design

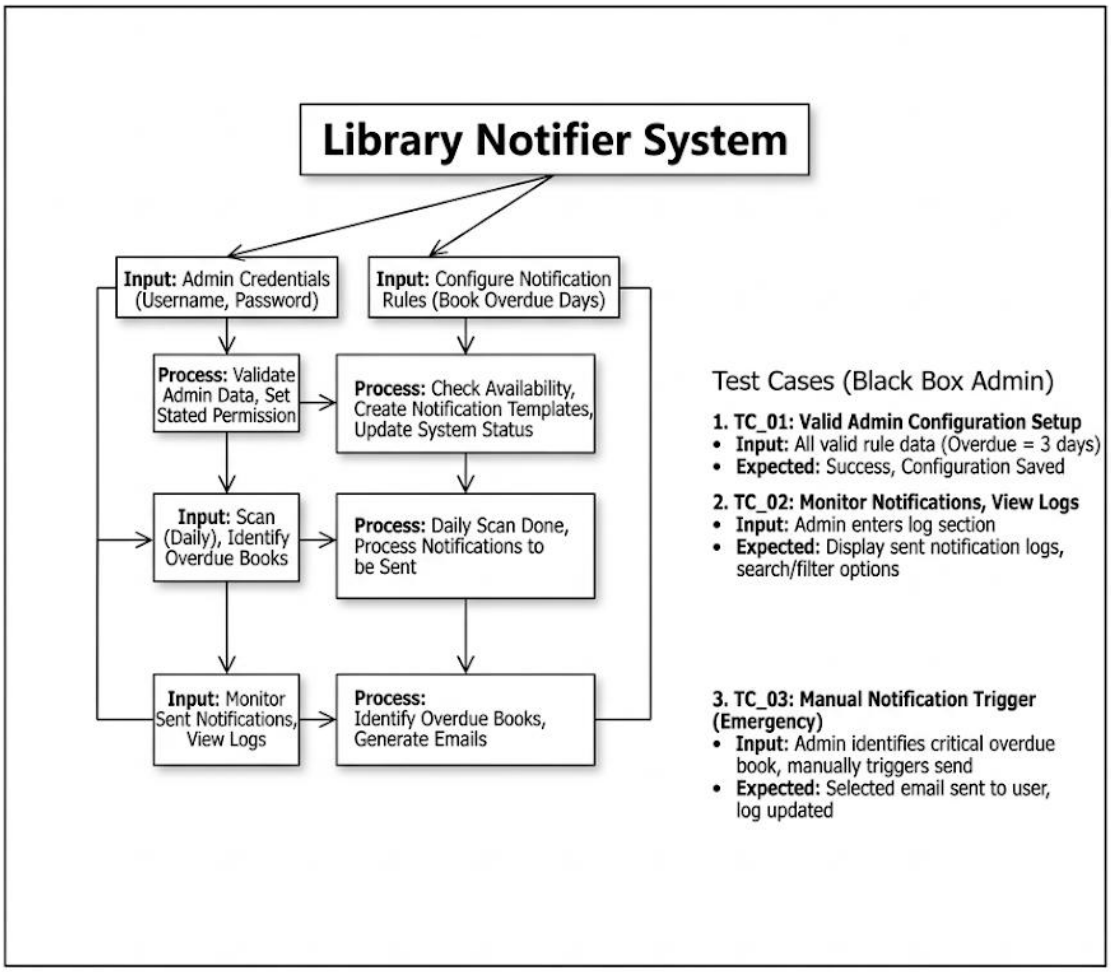


Fig 4.7 Test case design

The design of test cases defines scenarios, inputs, and expected outputs to verify that all system modules work correctly and meet requirements.

5 IMPLEMENTATION

5.1 Implementation Approaches:

The Library Notifier Web Application uses various web technologies for an easy-to-use platform for managing library books, student records, and the transactions related to books in the library. The focus/goal during this phase was to create a program that will allow an administrator to add books, manage student information, issue/return books, and keep track of overdue notifications.

Visual Studio Code was used as the development environment for the development of the application. The backend of the application was developed using Python and the Flask framework to implement the underlying business logic. HTML, CSS provided a graphical interface for the library operations and provided interactivity with the users when managing the operations. The information for the library is stored in the SQLite database; this includes books, students, issued books, and notifications.

The project is broken down into multiple modules for creating a well organized and maintainable system. The primary modules are the Book Management Module, Student Management Module, Book Issue and Return Module, Notification Module, and Admin Management Module. Each of the modules performs a different function such as storing book information, managing student information, logging issue/return transactions, and generating overdue book notifications.

5.2 Coding Details and Code Efficiency

The Library Notifier Web Application has been developed using the Python programming language and the Flask framework, and the frontend GUI has been created with a variety of technologies, such as HTML, CSS, and JavaScript. An SQLite database is utilized for the storage of data related to books and students as well as the issuance of books to students. The structure of the project is divided into a variety of modules, which helps create clean code that is easy to maintain. The modules that have been created for this project are book management, student management, book issuing and returning, and notification management, and the use of modules helps improve the readability of code, and therefore, makes the Library Notifier System easier to maintain and modify.

The routing of the Library Notifier Web Application and interaction between the frontend and backend logic are handled by the Flask framework. It gives the user options to create, modify, and retrieve records from the database using SQLite statements to perform the functions of creating, modifying, and retrieving records. The Library Notifier System will automatically check the return date of each issued book and will generate notifications for overdue books/concerning unpaid fines.

Using a structured modular coding approach allows for the Library Notifier System to operate efficiently and effectively, and help the developer to debug, maintain, and enhance the application in the future.

Example Code Snippet:

```
from flask import Flask, render_template
import sqlite3

app = Flask(__name__)

@app.route('/books')
def books():
    conn = sqlite3.connect("library.db")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM books")
    books = cursor.fetchall()
    conn.close()
    return render_template("books.html", books=books)
```

All of the records for the books that are located in the database will be retrieved and displayed to users of the Library Notifier System on the users books page using the books.html template.

5.2.1 Code Efficiency

The Library Notifier web application is designed in an efficient way by implementing web development using Python as a programming language with the use of small batch size of database queries through the Flask web Framework. Rather than loading all available records from the database, the application will only query needed data on available books, issued books, and overdue records. This type of design allows for the application to operate at an improved rate and user experience.

Additionally, it utilizes a modular programming approach by having separate modules handling the different functions of the library (books, students, issuing/returning books, and notifications). This improves the readability of the code, reduces code complexity, and will allow for simpler maintenance and future updates of the application.

The use of efficient front-end technologies (HTML/CSS/JavaScript) allows for a responsive and smooth user interface to be provided, while using Flask for back-end logic to communicate between the user interface and database. The use of organized code improves the performance and reliability of the entire system.

5.3 Testing Approach

Testing is a vital step during the development of Library Notifier web application. By conducting testing, you can test that your system is functioning properly and that all features

such as adding books, issuing books, returning books and displaying notifications will work correctly. In addition, testing can reveal bugs, errors or other problems that could arise after the application goes live in the real world library environment.

To complete the testing process, various test scenarios have been developed to ensure the system is functioning appropriately. For example, new book entries, student record entry into the system, issuing of books to students, returning of books from students, calculating late fees for overdue books, and displaying notifications from the system. Through testing processes, it is possible to ensure that each of the modules within your application works coherently with one another and delivers accurate results for effective library management.

5.3.1 Unit Testing

The Library Notifier Web Application's individual modules are tested in isolation to ensure the proper operation of the entire application. Each module book management, student management, issuing, returning, and notifying is tested independently, thus allowing the developer to verify that the expected outputs match the actual outputs for each of those parts.

The developer will test the book management module to ensure that they can add, modify, and find books from the database. The book issuing module will be tested to ensure that when a book is issued, there are still books available to be issued, and that the information regarding the book that was issued has been stored correctly in the database. The developer will also test that the system updates the quantity of each book correctly when an issued book is returned and that the system calculates any late fees that may be owed for overdue issued books.

The developer will also test the notification module to verify that notifications of overdue books will be sent when the book due dates are met. Each of the above unit tests will verify that each module does perform its designated job prior to combining the modules into the overall application.

5.3.2 Integrated Testing

Integrated Testing occurs after the Library Notifier Web Application is complete so that all of its modules can be combined into one complete system, which will include modules for managing books, managing students, issuing books and returning books, managing notifications, and performing database operations. At this point in time, the Integrated Library Notifier Web Application has one system that an administrator can use via a web interface to add books, store student information, issue books, return books, and check on overdue book notifications from the database.

Integrated Testing also ensures that all of the information regarding the Books, Students, and Transactions within the system are saved and retrieved correctly from the database, and that all the different modules can work together properly without any issues. Integrated Testing also ensures that the different modules can communicate with each other and are able to provide Library Administrators (an administrator) with a fully functional library management system.

5.3.3 Beta Testing

Beta Testing of the Library Notifier web application is conducted with various users to assess how well the system performs and how well it functions in an actual situation. For example, administrators or test-users run multiple library functions with the system and check for correct operations during its use.

Many scenarios are tested throughout beta testing, including the following: creating new books, maintaining student records, issuing books to students, returning books by students, and generating overdue notifications. This process ensures the system is capable of performing all operations when multiple users use it, with each function of the system being executed successfully through the web interface.

Beta testing provided documentation of the Library Notifier being able to accurately track the creation of book records, create and maintain the book inventory, process book issues/returns, and generate overdue notifications and applicable fines to patrons accurately. As a result, the Library Notifier application performed effectively as a library management system.

5.4 Modifications and Improvements

Many changes have been made to enhance the performance and usability of the Library Notifier Web Application during the testing phase. Improvements to the user interface of the system were made to help administrators manage books, students, and issued book records within a clean, organized layout.

The functionality of the system was also improved to ensure that the operations of issuing books and returning books were working properly, also that the changes made to the database were accurately executed. Additionally, improvements were made to the notification system to be able to accurately identify overdue books and display the fine calculations associated with the return of an overdue book.

Improvements have also been made to improve how the detail records of books, records of issued books, and notification alerts for the dashboard are displayed. Additionally, the structure of the code has been optimized to increase system performance and decrease the time it takes to retrieve data from the database. All of these changes have contributed to making the system more efficient, reliable and user-friendly in managing library operations.

5.5 Test Cases

1. Functionality Testing

• **Purpose:** To test that all features of the Library Notifier Web Application are working correctly.

• **Tests:**

- Test that the administrator can add, update, and view book records in the system.
- Test that books can be issued to students and returned successfully.
- Test that the system correctly displays issued books and available books.

2. Reliability Testing

• **Purpose:** To ensure that the system works consistently without errors.

• **Tests:**

- Test the system by performing multiple operations such as adding books, issuing books, and returning books repeatedly.
- Test that the system correctly updates the book quantity and maintains accurate records in the database.

3. Security Testing

• **Purpose:** To ensure that the system protects user and system data from unauthorized access.

• **Tests:**

- Test that only authorized users (admin) can access and manage library records.
- Test that login authentication works properly and prevents unauthorized access.

4. Performance Testing

• **Purpose:** To test the performance and response time of the system.

• **Test Cases:**

- Test the system response time when multiple operations such as adding or issuing books are performed.
- Test that the system quickly retrieves and displays book records and overdue notifications from the database.

6 RESULTS AND DISCUSSION

6.1 Test Reports

Testing results indicate that the Library Notifier Web Application has met all objectives for Library Management operations, enabling the Administrator to carry out all major functions related to Library Management, including management of Book Records, storage of Student data, issuing of Books and processing returns of Books with maximum efficiency.

As a result, the Library Notifier Web Application accurately manages Library Information through its ability to Create, Read, Update and Delete (CRUD) data in the database. The Library Notifier Web Application accurately records all transactions for issuing and returning Books, maintains correct availability of Books in the Library Notifier Web Application Database, provides users with notifications of overdue Books, and calculates and displays late return fines when Books are returned after their due dates.

The overall results of testing confirm that the Library Notifier Web Application functions properly under normal conditions and provides accurate results with respect to Books, Students, and Issued Records. The Library Notifier Web Application is reliable and effective in enhancing the efficiency of Library Management.

6.2 User Documentation

User documentation provides guidelines on how to execute and use the Library Notifier Web Application.

Steps for Executing the System:

• Install the required software such as Python and the Flask framework.
• Install the necessary libraries mentioned in the requirements.txt file.
• Open the project folder using Visual Studio Code or any other code editor.
• Run the main application file (for example app.py) to start the server.
• Open a web browser and access the system using the local host link (e.g., http://127.0.0.1:5000).
• Log in to the system as an administrator to access the dashboard.

The system interface will display different options such as adding books, managing student records, issuing books, returning books, and viewing notifications. The administrator can add new books, store student information, and track issued books through the dashboard. If a book is returned after the due date, the system will automatically display an overdue notification and calculate the fine accordingly.

7 CONCLUSION

7.1 Conclusion

The Library Notifier web app was created to make it easier and simpler to manage library operations. The capabilities of this system include keeping a record of books, managing student information, and processing book issuing/returning. With the automation of these tasks, there are fewer mistakes and problems involved with managing a library on paper.

One of the primary features of the system is a notification system that helps library staff know when books are late and calculate the fine amount due for books returned after the due date. This helps library staff track late returning books as well as ensure adherence to all library rules. This helps library staff manage their books more effectively.

The application was built using Python for its backend and using Flask as the web application framework. HTML, CSS and JavaScript were used to build the frontend of the application. The SQLite database is used to store and manage all of the data for the application including books, students, and transaction data. The system is designed to be very easy for staff to use while performing various library management activities.

The new system enhances the efficiency, accuracy, and reliability of library business operations. Effectively managing library information enables multiple departments to access this information in one location and reduces the time needed to maintain records. Future upgrades will increase functionality (e.g., emailing, searching online for books, multi-user access) and improve the user experience for people using the system.

7.1.1 Significance of the system

The Library Notifier Web Application is used as an online digital record-keeping system for managing library activities such as storing information on books, storing information on borrowers, and tracking transactions (loan and return of books). It helps streamlining library operations through the reduction of clerical work for the administrators of the library enabling them to operate in a more organized and efficient manner.

7.2 Limitations of the system

The system is designed mainly for a single administrator and does not support multiple user roles with different access levels. Additionally, the system may require proper system setup and configuration to run smoothly on different devices.

7.3 FUTURE SCOPES OF THE PROJECT

➢ **Email and SMS Notifications:** The system can be improved by adding email or SMS notifications to automatically remind users about due dates and overdue books.

➢ **Online Book Reservation:** Users can be allowed to search and reserve books online before visiting the library.

➢ **Mobile Application Development:** The system can be developed as a mobile application for Android and iOS devices to make library services more accessible.

➢ **Multi-User Access:** The system can support multiple user roles such as admin, librarian, and students with different access permissions.

➢ **Online Fine Payment:** An online payment feature can be added so that students can pay overdue fines through digital payment methods.

➢ **Advanced Search System:** The system can include advanced search filters to help users easily find books by title, author, or category.

➢ **Analytics and Reports:** An admin dashboard with reports and analytics can be added to track library usage, overdue records, and popular books.

➢ **Cloud Database Integration:** The system can be connected to a cloud database to allow access from different locations and improve data backup and security.

➢ **Improved Security Features:** Additional security measures such as encrypted login and stronger authentication methods can be implemented to protect library data.

References

- Aggarwal, C. C. (2014*).* Data Mining*:* The Textbook. Springer.

- Ajayi, N. A., & Okunlola, A. A. (2005). Students' perception of fine increases for overdue library books. Library Philosophy and Practice.

- Kumar, V., & Singh, R. (2017). Automated library management system with fine calculation and notification services. International Journal of Computer Applications, 164(9), 22–27.

- Singh, P., & Sharma, A. (2019). Design of a library due-date reminder and fine management system. International Journal of Engineering and Advanced Technology, 8(6), 245–250.

- GeeksforGeeks. (2022). How to build a library management system using python and MySQL.

- Google Scholar. (2015–2023). Research papers on smart library systems, SMS/email reminders, and automated fine calculation.

- ResearchGate. (2018). Content-based messaging model for library information systems.

- CodeProject. (2020). Developing a notification-based library management application. Retrieved from

- W3Schools. (2023). Tutorials on database integration, web notifications, and user management for online systems.

- Mishra, S., & Patel, D. (2016). Library automation and due-date notifier system using email gateway.

GLOSSARY

Term	Meaning
Admin	Person who manages the library system and records
Book	A resource available in the library for students to borrow
Book Issue	Process of giving a book to a student for a specific period
Book Return	Process of returning a borrowed book to the library
Due Date	The date before which the issued book must be returned
Overdue Book	A book that is not returned before the due date
Fine	Penalty amount charged for returning a book after the due date
Notification	Alert generated by the system for overdue books
Database	Storage used to keep records of books, students, and transactions

APPENDIX A

CODE:

app.py

```
from flask import Flask, render_template, request, redirect, flash, session
import sqlite3
from datetime import datetime, timedelta
from flask import jsonify, request


app = Flask(__name__)
app.secret_key = "advanced_library_secret"
# ================ DATABASE ================
def get_db():
def create_tables():
create_tables()
# ================ LOGIN ================
@app.route("/", methods=["GET", "POST"])
def login():
@app.route("/logout")
def logout():
# ================ DASHBOARD ================
@app.route("/dashboard")
def dashboard():
@app.route('/books', methods=['GET','POST'])
def books():
# ================ ISSUE ================
@app.route("/issue/<int:id>", methods=["POST"])
def issue(id):
# ================ NOTIFICATIONS ================
```

```
@app.route("/notifications")
def notifications():
# ================ ISSUE BOOK PAGE ================
@app.route("/issue-book", methods=["GET", "POST"])
def issue_book():
# ================ ISSUED BOOKS ================
@app.route("/issued-books")
def issued_books_list():
#====================Student=================
@app.route('/students', methods=['GET', 'POST'])
def students():
@app.route('/get-student')
def get_student():
#==============Return button==============#
@app.route("/return_book/<int:id>")
def return_book_action(id):
if __name__ == "__main__":
    app.run(debug=True)
```

Index.html

```
{% extends "base.html" %}
{% block content %}
<h2 class="mb-4">Available Books</h2>
<table class="table table-hover shadow">
    <thead class="table-dark">
        <tr>
            <th>Title</th>
            <th>Author</th>
            <th>Status</th>
            <th>Action</th>
```

```
        </tr>
    </thead>
    <tbody>
        {% for book in books %}
        <tr>
            <td>{{ book.title }}</td>
            <td>{{ book.author }}</td>
            <td>
                {% if book.status == "Available" %}
                    <span class="badge bg-success">Available</span>
                {% else %}
                    <span class="badge bg-danger">Issued</span>
                {% endif %}
            </td>
            <td>
                {% if book.status == "Available" %}
                    <a href="/issue/{{ book.id }}" class="btn btn-primary btn-sm">Issue</a>
                {% else %}
                    <button class="btn btn-secondary btn-sm" disabled>Issued</button>
                {% endif %}
            </td>
        </tr>
        {% endfor %}
    </tbody>
</table>
{% endblock %}
```

Login.html

```
<!DOCTYPE html>
<html>
```

```
<head>
  <title>Library Login</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      background: linear-gradient(135deg, #667eea, #764ba2);
      height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
    }
    .login-card {
      width: 400px;
      padding: 30px;
      border-radius: 15px;
      background: white;
      box-shadow: 0 10px 30px rgba(0,0,0,0.2);
    }
  </style>
</head>
<body>
<div class="login-card">
  <h3 class="text-center mb-4">📚 Library Admin Login</h3>
  {% with messages = get_flashed_messages() %}
    {% if messages %}
      <div class="alert alert-danger">
        {{ messages[0] }}
      </div>
    {% endif %}
  {% endwith %}
```

```
    <form method="POST">
        <div class="mb-3">
            <label>Username</label>
            <input type="text" name="username" class="form-control" required>
        </div>
        <div class="mb-3">
            <label>Password</label>
            <input type="password" name="password" class="form-control" required>
        </div>
        <button class="btn btn-primary w-100">Login</button>
    </form>
    <div class="text-center mt-3 text-muted">
        Default: admin / admin123
    </div>
</div>
</body>
</html>
```

Notification.html

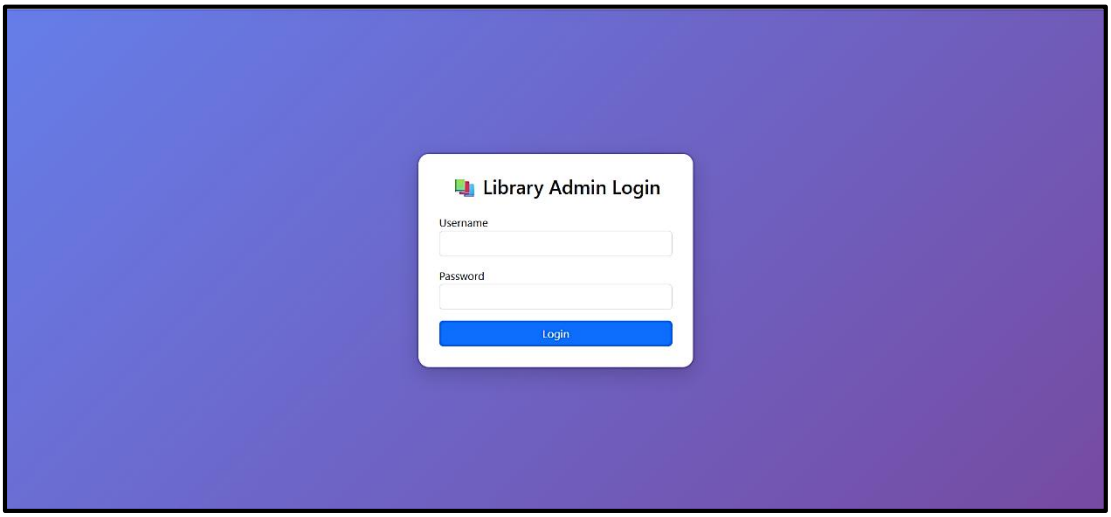
```
<!DOCTYPE html>
<html>
<head>
    <title>Overdue Notifications</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
    <nav class="navbar navbar-dark bg-dark px-4">
    <span class="navbar-brand mb-0 h1">🔔 Notification</span>
</nav>
```

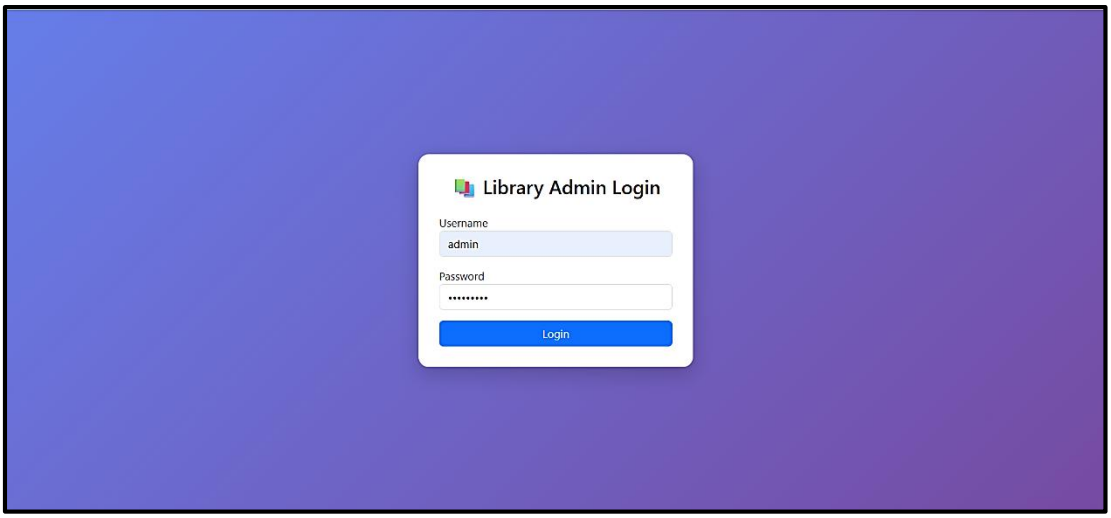
```
<div class="container mt-5">
  <div class="card shadow-lg">
    <div class="card-header bg-danger text-white">
      <h4 class="mb-0">□ Overdue Notifications</h4>
    </div>
    <div class="card-body">
      {% if notifications %}
        <div class="row">
          {% for note in notifications %}
          <div class="col-md-6 mb-4">
            <div class="card border-danger shadow-sm h-100">
              <div class="card-body">
                <h5 class="card-title text-danger fw-bold">
                  📚 {{ note.title }}
                </h5>
                <p class="mb-2">
                  <strong>Borrowed By:</strong> {{ note.student }}
                </p>
                <div class="mt-2">
  <span class="badge bg-danger fs-6">
    □ Overdue by {{ note.days }} days
  </span>
</div>
<div class="mt-2">
  <span class="badge bg-dark fs-6">
    💰 Fine: ₹ {{ note.fine }}
  </span>
</div>
              </div>
            </div>
```

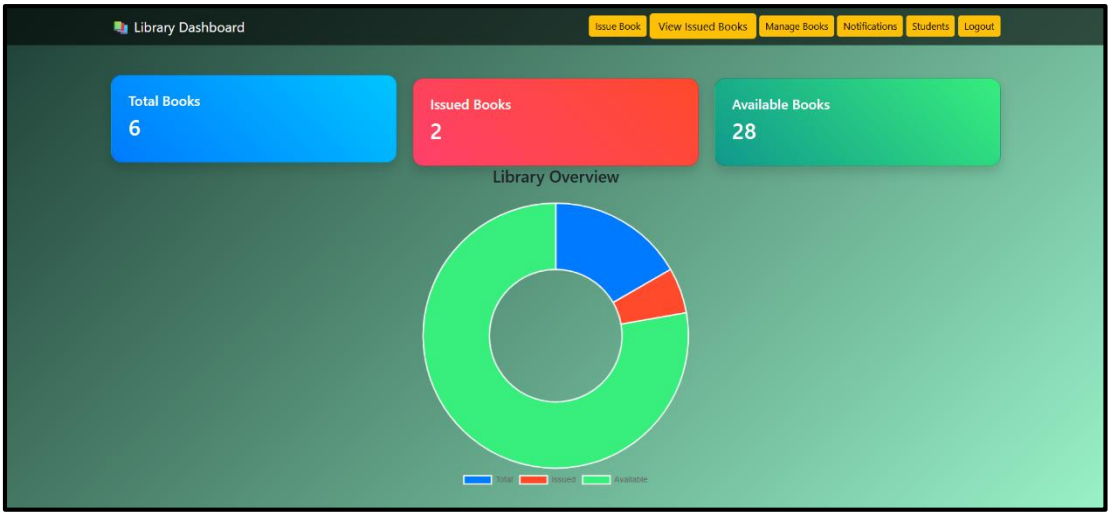
```
                </div>
                {% endfor %}
            </div>
        {% else %}
            <div class="alert alert-success text-center">
                🎉 No overdue books right now!
            </div>
        {% endif %}
    </div>
  </div>
</div>
<style>
body{
  min-height:100vh;
  background: linear-gradient(135deg, #f0f4f3, #dcefe6);
  background-attachment: fixed;
}
</style>
</body>
</html>
```

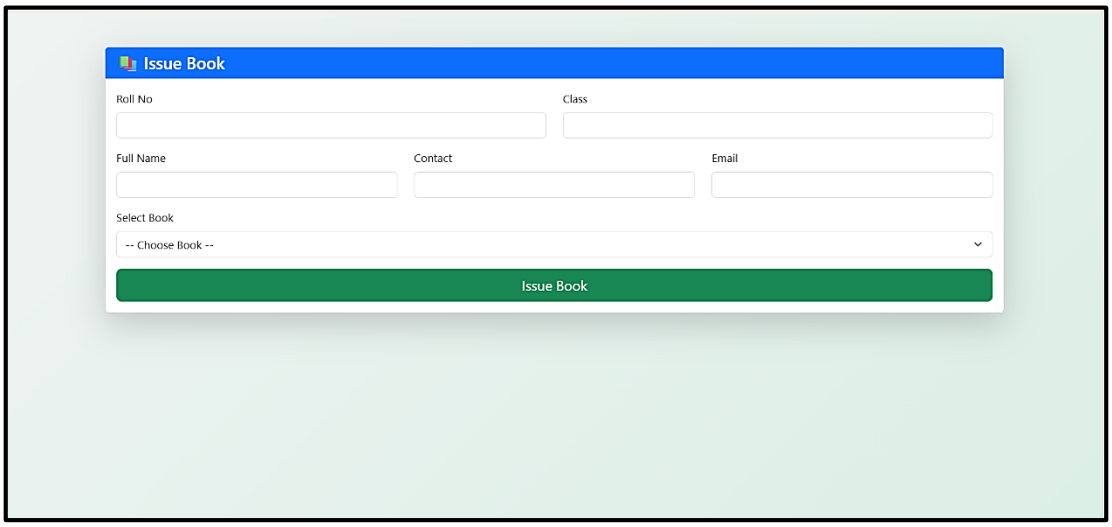
APPENDIX B

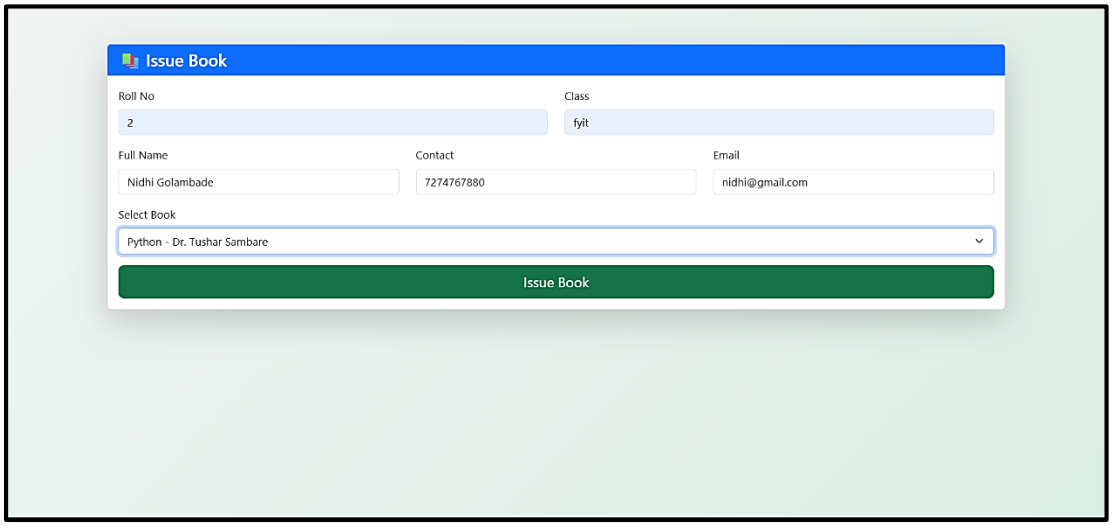
Output

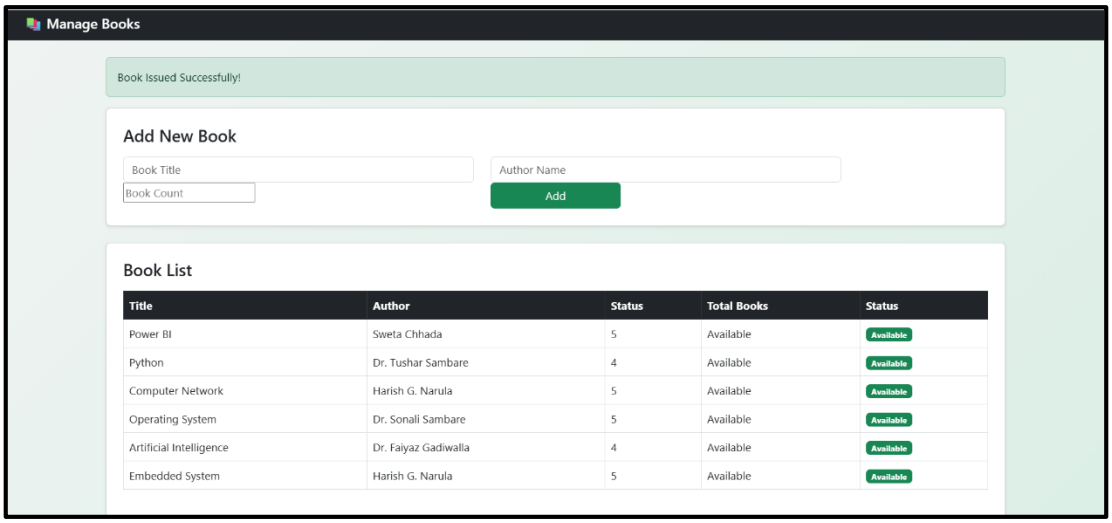


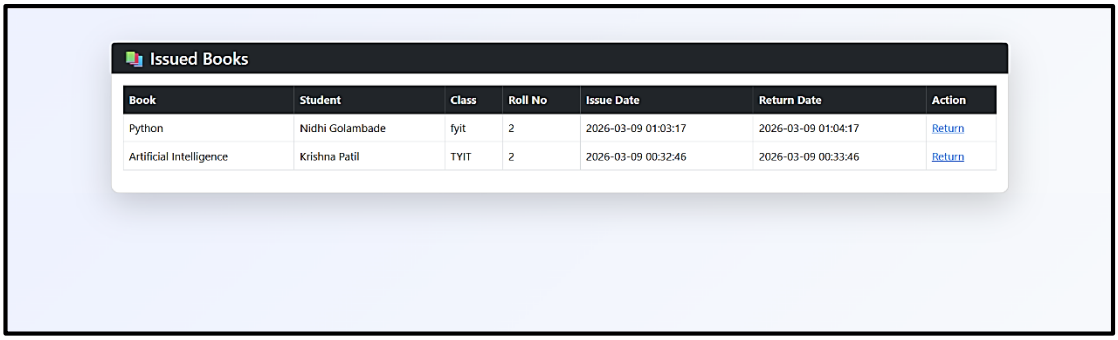


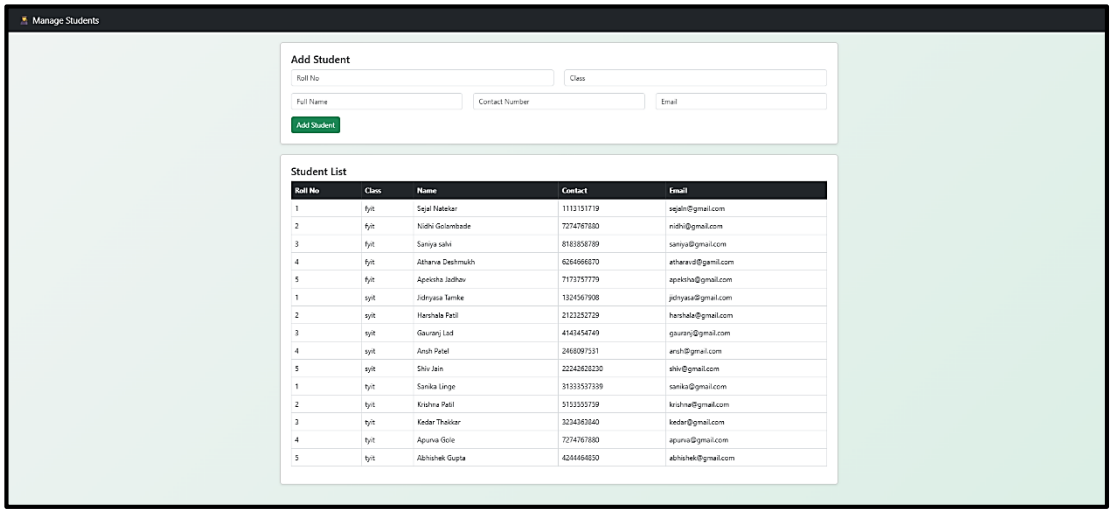


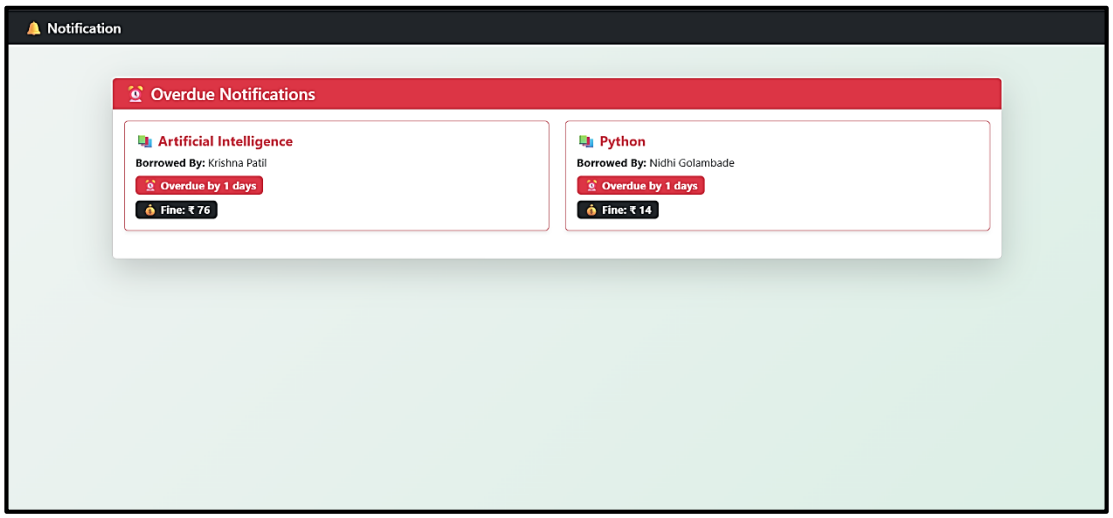

Issue Book
Roll No
Class
Full Name
Contact
Email
Select Book
-- Choose Book --
Issue Book


Issue Book
Roll No
2
Class
fyit
Full Name
Nidhi Golambade
Contact
7274767880
Email
nidhi@gmail.com
Select Book
Python - Dr. Tushar Sambare
Issue Book


Manage Books
Book Issued Successfully!
Add New Book
Book Title
Author Name
Book Count
Add
Book List
Title
Author
Status
Total Books
Status
Power BI
Sweta Chhada
5
Available
Available
Python
Dr. Tushar Sambare
4
Available
Available
Computer Network
Harish G. Narula
5
Available
Available
Operating System
Dr. Sonali Sambare
5
Available
Available
Artificial Intelligence
Dr. Faiyaz Gadiwalla
4
Available
Available
Embedded System
Harish G. Narula
5
Available
Available


Issued Books
Book
Student
Class
Roll No
Issue Date
Return Date
Action
Python
Nidhi Golambade
fyit
2
2026-03-09 01:03:17
2026-03-09 01:04:17
Return
Artificial Intelligence
Krishna Patil
TYIT
2
2026-03-09 00:32:46
2026-03-09 00:33:46
Return


Manage Students
Add Student
Roll No
Class
Full Name
Contact Number
Email
Add Student
Student List
Roll No | Class | Name | Contact | Email
1 | fyit | Sejal Natekar | 1113151719 | sejaln@gmail.com
2 | fyit | Nidhi Golambade | 7274767880 | nidhi@gmail.com
3 | fyit | Saniya sahi | 8183858789 | saniya@gmail.com
4 | fyit | Atharva Deshmukh | 6264666870 | atharavd@gamil.com
5 | fyit | Apeksha Jadhav | 7173757779 | apeksha@gmail.com
1 | syit | Jidnyasa Tamke | 1324567908 | jidnyasa@gmail.com
2 | syit | Harshala Patil | 2123252729 | harshala@gmail.com
3 | syit | Gauranj Lad | 4143454749 | gauranj@gmail.com
4 | syit | Ansh Patel | 2468097531 | ansh@gmail.com
5 | syit | Shiv Jain | 22242828230 | shiv@gmail.com
1 | tyit | Sanika Linge | 31333537339 | sanika@gmail.com
2 | tyit | Krishna Patil | 5153355739 | krishna@gmail.com
3 | tyit | Kedar Thakkar | 3234363640 | kedar@gmail.com
4 | tyit | Apurva Gole | 7274767880 | apurva@gmail.com
5 | tyit | Abhishek Gupta | 4244464830 | abhishek@gmail.com


Notification
Overdue Notifications
Artificial Intelligence
Borrowed By: Krishna Patil
Overdue by 1 days
Fine: ₹ 76
Python
Borrowed By: Nidhi Golambade
Overdue by 1 days
Fine: ₹ 14